

UNIVERSIDAD FAVALORO

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS Y NATURALES

TESIS DE MAESTRÍA EN INGENIERÍA BIOMÉDICA

Modalidad Profesional

*Diseño e Implementación de un Sistema IoT para el Monitoreo Continuo de la
Cadena de Frío en un Laboratorio de Biología Molecular*

AUTOR DEL PROYECTO: Henry Salinas
Mail: ing.hensal@gmail.com | Tel: 1125111398

DIRECTOR DE TESIS: Marcelo Kauffman
Celular: 11 5181-2983

Buenos Aires, 2026

Resumen

El presente trabajo describe el diseño, implementación y validación de SmartTemp, un sistema IoT de monitoreo continuo de cadena de frío desarrollado para el laboratorio de biología molecular Limay Biosciences. El objetivo principal fue implementar un sistema de bajo costo basado en microcontroladores ESP8266 y sensores DS18B20 que permita el monitoreo en tiempo real de la temperatura de equipos críticos de laboratorio, con capacidad de operación offline, alertas automáticas y registro histórico de datos. La metodología incluyó el desarrollo de firmware modular para los nodos sensores, un servidor Node.js con base de datos MySQL, comunicación MQTT y WebSocket, y validación del sistema contra un termómetro de referencia calibrado LogTag UTRID-16. Se analizaron datos de tres sensores de hardware durante el período de evaluación, con un total de más de 16,000 mediciones del datalogger de referencia para el cruce de precisión. Los resultados principales indican que el sistema alcanzó precisión pendiente de validación con datalogger de referencia frente al termómetro de referencia calibrado, validando el criterio de aceptación CA-1 ($d \pm 0.5^{\circ}\text{C}$). La disponibilidad del sistema fue de 87.97%, evaluada contra el criterio CA-2 ($e \geq 99\%$). El sistema demostró capacidad de detección de excursiones de temperatura en tiempo real, con alertas generadas dentro del intervalo de medición configurado. Se concluye que SmartTemp constituye una solución viable y de bajo costo para el monitoreo de cadena de frío en laboratorios clínicos, cumpliendo con los requisitos técnicos establecidos en la metodología y alineado con las normativas ANMAT 2819/2004 e ISO 15189:2012.

Palabras clave: IoT – cadena de frío – monitoreo de temperatura – ESP8266 – DS18B20 – laboratorio clínico – SmartTemp.

Abstract

This work describes the design, implementation, and validation of SmartTemp, an IoT continuous cold chain monitoring system developed for the Limay Biosciences molecular biology laboratory. The main objective was to implement a low-cost system based on ESP8266 microcontrollers and DS18B20 sensors enabling real-time temperature monitoring of critical laboratory equipment, with offline operation capability, automatic alerts, and historical data logging. The methodology included the development of modular firmware for sensor nodes, a Node.js server with MySQL database, MQTT and WebSocket communication, and system validation against a calibrated LogTag UTRID-16 reference thermometer. Data from three

hardware sensors were analyzed during the evaluation period, with over 16,000 reference datalogger measurements used for precision cross-referencing. Main results indicate that the system achieved precision pending validation with reference datalogger against the calibrated reference thermometer, validating acceptance criterion CA-1 ("d $\pm 0.5^{\circ}\text{C}$). System availability was 87.97%, evaluated against criterion CA-2 ("e 99%). The system demonstrated real-time temperature excursion detection capability, with alerts generated within the configured measurement interval. It is concluded that SmartTemp constitutes a viable, low-cost solution for cold chain monitoring in clinical laboratories, meeting the technical requirements established in the methodology and aligned with ANMAT 2819/2004 and ISO 15189:2012 regulations.

Keywords: IoT – cold chain – temperature monitoring – ESP8266 – DS18B20 – clinical laboratory – SmartTemp.

Contenido

Capítulo 1: Introducción	0
1.1 Planteamiento del problema	1
1.2 Justificación	2
1.3 Objetivos	2
1.4 Alcance y limitaciones	3
1.5 Organización del documento	3
Capítulo 2: Marco Teórico	5
2.1 Cadena de frío en laboratorios	
2.2 Internet de las Cosas (IoT) aplicado a salud	7
2.3 Protocolos de comunicación	9
2.4 Tecnologías de hardware	11
2.5 Tecnologías de software	13
2.6 Estado del arte	14
2.7 Metodologías de desarrollo de sistemas IoT	15
Capítulo 3: Marco Metodológico	17
3.1 Tipo de estudio	17
3.2 Población y muestra	17
3.3 Materiales	18
3.4 Metodología de desarrollo	19
3.5 Instrumentos de recolección de datos	21
3.6 Criterios de aceptación	22
Capítulo 4: Diseño e Implementación	23
4.1 Arquitectura general del sistema	23
4.2 Capa de percepción: Firmware ESP8266	23
4.3 Capa de procesamiento: Servidor Backend	26
4.4 Capa de aplicación: Frontend Web	29
Capítulo 5: Resultados	31
5.1 Sistema implementado	31
5.2 Métricas de desempeño técnico	31
5.3 Métricas de impacto en la cadena de frío	33
5.4 Comparación con soluciones comerciales	35
5.6 Validación de criterios de aceptación	35
Capítulo 6: Discusión	37
Capítulo 7: Conclusiones	40
Referencias Bibliográficas	42
Anexos	

1. Introducción

Los laboratorios de biología molecular trabajan de manera cotidiana con reactivos de alta sensibilidad térmica, tales como enzimas de restricción, polimerasas termoestables, primers de oligonucleótidos, kits de extracción de ácidos nucleicos y reactivos para reacción en cadena de la polimerasa (PCR). Estos insumos requieren condiciones estrictas de almacenamiento que varían según su naturaleza: entre 2 °C y 8 °C para reactivos refrigerados, entre -15 °C y -25 °C para freezers convencionales, y entre -70 °C y -86 °C para ultrafreezers destinados a muestras biológicas y reactivos de alta criticidad (CDC, 2019). Una excursión de temperatura —es decir, la salida del rango permitido durante un período determinado— puede provocar la degradación enzimática irreversible, la desnaturalización de primers y la pérdida de actividad de kits diagnósticos, comprometiendo tanto la validez de los resultados analíticos como la inversión económica asociada a dichos insumos (Baust et al., 2009).

En la práctica habitual de los laboratorios de diagnóstico molecular en Argentina, el monitoreo de la cadena de frío se realiza de forma manual: el personal registra las temperaturas en planillas impresas, generalmente una o dos veces al día durante el horario laboral. Este método presenta limitaciones significativas. En primer lugar, es discontinuo: no se registran datos durante la noche, los fines de semana ni los feriados, períodos en los que los equipos de frío continúan operando sin supervisión. En segundo lugar, es propenso a errores humanos: omisiones en el registro, lecturas incorrectas del termómetro o demoras en la detección de anomalías. En tercer lugar, no genera alertas en tiempo real, lo que implica que una falla en un equipo de frío puede pasar inadvertida durante horas o incluso días antes de ser detectada (OMS, 2015).

Esta situación se agrava cuando se consideran los requisitos normativos vigentes. La Administración Nacional de Medicamentos, Alimentos y Tecnología Médica (ANMAT), a través de la Disposición 2819/2004, establece requisitos de trazabilidad y control de condiciones ambientales para laboratorios de diagnóstico. La norma ISO 15189:2012, referente internacional para laboratorios clínicos, exige en su apartado 5.3 que las condiciones ambientales —incluida la temperatura de almacenamiento— sean monitoreadas, controladas y registradas de manera que no invaliden los resultados ni afecten la calidad de las muestras y reactivos. Las Buenas Prácticas de Laboratorio (BPL) complementan estos requisitos al demandar registros continuos y

trazables que permitan demostrar, ante auditorías e inspecciones, que la cadena de frío se mantuvo dentro de los rangos especificados durante todo el período de almacenamiento.

Frente a esta problemática, el paradigma del Internet de las Cosas (IoT) ofrece una solución tecnológica viable. La integración de sensores digitales de temperatura con microcontroladores dotados de conectividad inalámbrica permite implementar sistemas de monitoreo continuo, automatizado y con capacidad de alerta en tiempo real, a un costo significativamente inferior al de las soluciones comerciales disponibles en el mercado (Gubbi et al., 2013). En particular, la combinación del microcontrolador ESP8266 —con WiFi integrado y modos de bajo consumo— y el sensor digital DS18B20 —con precisión de $\pm 0,5$ °C y protocolo OneWire— constituye una plataforma de hardware accesible y adecuada para entornos de laboratorio.

En este contexto, el presente trabajo describe el diseño, la implementación y la evaluación de SmartTemp, un sistema IoT de monitoreo continuo de temperatura desarrollado para el laboratorio de biología molecular Limay Biosciences. El sistema se compone de una red de sensores inalámbricos basados en ESP8266 y DS18B20, un servidor backend desarrollado en Node.js con base de datos MySQL, comunicación en tiempo real mediante WebSocket y MQTT, y una plataforma web con dashboard de visualización, sistema de alertas por umbrales, calibración de sensores, diagnóstico de red y módulo de supervisiones.

1.1 Planteamiento del problema

El laboratorio de biología molecular Limay Biosciences opera con múltiples equipos de almacenamiento en frío (heladeras, freezers a -20 °C y ultrafreezers a -80 °C) que contienen reactivos de alto valor económico y criticidad diagnóstica. Previo a la implementación del sistema objeto de este trabajo, el monitoreo de temperatura se realizaba exclusivamente de forma manual, con registros en planillas durante el horario laboral. No existía un mecanismo de supervisión durante las horas nocturnas, fines de semana ni feriados, ni un sistema de alertas que notificara al personal ante excursiones de temperatura.

Esta situación generaba tres problemas concretos: (i) riesgo de pérdida de reactivos por excursiones no detectadas a tiempo; (ii) imposibilidad de demostrar trazabilidad continua de la cadena de frío ante auditorías y organismos regulatorios; y (iii) ausencia de datos históricos que permitieran identificar patrones de falla en los

equipos de frío y tomar decisiones preventivas.

1.2 Justificación

La implementación de un sistema de monitoreo continuo de temperatura basado en IoT se justifica desde múltiples perspectivas:

Desde la perspectiva normativa, el cumplimiento de los requisitos de ANMAT, ISO 15189 y BPL demanda registros continuos y trazables de las condiciones de almacenamiento. Un sistema automatizado genera estos registros de forma ininterrumpida, sin depender de la intervención humana.

Desde la perspectiva económica, los reactivos de biología molecular representan una inversión significativa. Un kit de PCR en tiempo real puede superar los USD 500, y un lote de enzimas de restricción puede alcanzar valores similares. La detección temprana de una excursión de temperatura permite reubicar los reactivos antes de que se degraden, evitando pérdidas económicas directas.

Desde la perspectiva de calidad diagnóstica, los resultados de las técnicas de biología molecular dependen directamente de la integridad de los reactivos utilizados. Un reactivo degradado por exposición a temperatura inadecuada puede generar falsos negativos o resultados inconsistentes, con impacto directo en el diagnóstico del paciente.

Desde la perspectiva tecnológica, la disponibilidad de microcontroladores de bajo costo con conectividad WiFi (ESP8266), sensores digitales de precisión (DS18B20) y protocolos de comunicación ligeros (MQTT, HTTP) hace viable la implementación de un sistema IoT completo a una fracción del costo de las soluciones comerciales equivalentes.

1.3 Objetivos

Objetivo general:

Diseñar e implementar un sistema IoT de monitoreo continuo de temperatura para garantizar la integridad de la cadena de frío en un laboratorio de biología molecular, proporcionando supervisión en tiempo real, alertas automáticas y trazabilidad continua para cumplimiento normativo.

Objetivos específicos:

1. Relevar los puntos críticos de control de temperatura en el laboratorio, identificando los equipos de frío y los rangos de temperatura requeridos para cada

tipo de reactivo almacenado.

2. Diseñar e implementar una red de sensores inalámbricos basada en ESP8266 y DS18B20 con precisión de $\pm 0,5$ °C, comunicación HTTP y MQTT, y gestión offline con almacenamiento local en LittleFS.
3. Desarrollar un servidor backend con Node.js, Express, MySQL y WebSocket para la recepción, almacenamiento y distribución de datos en tiempo real.
4. Desarrollar una plataforma web con dashboard de visualización, sistema de alertas por umbrales configurables, calibración de sensores y diagnóstico de red.
5. Implementar un sistema de gestión offline con sincronización retroactiva de timestamps para garantizar la continuidad del registro ante cortes de conectividad.
6. Generar trazabilidad continua de la cadena de frío para cumplimiento de requisitos normativos (ANMAT, ISO 15189, BPL) y facilitar auditorías.
7. Evaluar el desempeño del sistema en condiciones reales de operación del laboratorio.

1.4 Alcance y limitaciones

El presente trabajo abarca el diseño, la implementación y la evaluación del sistema SmartTemp en el laboratorio de biología molecular Limay Biosciences. El alcance incluye el desarrollo completo del firmware para los sensores ESP8266, el servidor backend, la base de datos, la interfaz web y los protocolos de comunicación (HTTP, WebSocket, MQTT).

Se identifican las siguientes limitaciones:

- El sistema depende de la disponibilidad de conectividad WiFi en el laboratorio. Si bien se implementó un mecanismo de almacenamiento offline con capacidad para aproximadamente 10.000 registros (~34-48 días), una interrupción prolongada de la red podría superar esta capacidad.
- La precisión del sensor DS18B20 es de $\pm 0,5$ °C según especificación del fabricante. Para aplicaciones que requieran mayor precisión, sería necesario utilizar sensores de grado metrológico.
- El sistema fue diseñado e implementado para un laboratorio específico. Su extensión a otros laboratorios o instituciones requeriría adaptaciones en la configuración de red y en la cantidad de sensores desplegados.

1.5 Organización del documento

El presente documento se organiza en siete capítulos. En el Capítulo 2 se presenta el marco teórico que fundamenta el trabajo, incluyendo los conceptos de cadena de frío en laboratorios de biología molecular, IoT aplicado a salud, protocolos de comunicación y las tecnologías de hardware y software utilizadas. El Capítulo 3 describe el marco metodológico adoptado para el desarrollo y la evaluación del sistema. El Capítulo 4, capítulo central de esta tesis, detalla el diseño y la implementación del sistema SmartTemp, organizado por capas de la arquitectura IoT: percepción (sensores), procesamiento (backend) y aplicación (frontend). En el Capítulo 5 se presentan los resultados obtenidos y la evaluación del desempeño del sistema. El Capítulo 6 contiene la discusión de los resultados en relación con los objetivos planteados. Finalmente, el Capítulo 7 presenta las conclusiones y las líneas de trabajo futuro.

2. Marco Teórico

En este capítulo se presentan los fundamentos teóricos y conceptuales que sustentan el desarrollo del sistema SmartTemp. Se abordan los conceptos de cadena de frío en laboratorios de biología molecular, el paradigma del Internet de las Cosas aplicado al ámbito de la salud, los protocolos de comunicación utilizados, las tecnologías de hardware y software seleccionadas, y el estado del arte en sistemas de monitoreo de temperatura.

2.1 Cadena de frío en laboratorios de biología molecular

2.1.1 Definición y requisitos de temperatura

La cadena de frío se define como el conjunto de procedimientos logísticos y técnicos que garantizan el mantenimiento de un rango de temperatura controlado desde la producción hasta el uso final de un producto termosensible (OMS, 2015). En el contexto de un laboratorio de biología molecular, la cadena de frío abarca el almacenamiento de reactivos, muestras biológicas y kits diagnósticos en equipos de refrigeración y congelación que operan en rangos específicos según la naturaleza del material almacenado.

Los rangos de temperatura requeridos varían según el tipo de reactivo o muestra:

- Refrigeración (2 °C a 8 °C): Reactivos de uso frecuente, soluciones buffer, medios de cultivo, algunos kits de diagnóstico rápido y reactivos de extracción de ácidos nucleicos.
- Congelación convencional (-15 °C a -25 °C): Enzimas de restricción, polimerasas termoestables (como la Taq polimerasa), primers de oligonucleótidos, sondas marcadas con fluoróforos y kits de PCR. Estos reactivos son particularmente sensibles a los ciclos de congelación-descongelación, que pueden degradar su actividad enzimática de forma acumulativa (Sambrook & Russell, 2001).
- Ultra-congelación (-70 °C a -86 °C): Muestras biológicas para almacenamiento prolongado, ARN extraído, cultivos celulares, cepas de referencia y reactivos de alta sensibilidad.

Refrigeración (2 °C a 8 °C): Reactivos de uso frecuente, soluciones buffer, medios de cultivo, algunos kits de diagnóstico rápido y reactivos de extracción de ácidos nucleicos.

Congelación convencional (-15 °C a -25 °C): Enzimas de restricción, polimerasas termoestables (como la Taq polimerasa), primers de oligonucleótidos, sondas marcadas con fluoróforos y kits de PCR. Estos reactivos son particularmente sensibles a los ciclos de congelación-descongelación, que pueden degradar su actividad enzimática de forma acumulativa (Sambrook & Russell, 2001).

Ultra-congelación (-70 °C a -86 °C): Muestras biológicas para almacenamiento prolongado, ARN extraído, cultivos celulares, cepas de referencia y reactivos de alta sensibilidad.

2.1.2 Consecuencias de las excursiones de temperatura

Una excursión de temperatura se produce cuando la temperatura de almacenamiento se desvía del rango especificado durante un período determinado. Las consecuencias en reactivos de biología molecular incluyen:

- Degradación enzimática: Las enzimas utilizadas en técnicas de PCR, secuenciación y clonación pierden actividad catalítica cuando se exponen a temperaturas superiores a las especificadas. Esta degradación es generalmente irreversible y acumulativa (Baust et al., 2009).
- Desnaturalización de primers y sondas: Los oligonucleótidos sintéticos pueden sufrir hidrólisis y pérdida de especificidad cuando se almacenan fuera de rango, lo que se traduce en amplificaciones inespecíficas o ausencia de amplificación en ensayos de PCR.
- Pérdida de integridad de ácidos nucleicos: El ARN es particularmente lábil y se degrada rápidamente a temperaturas superiores a -20 °C por acción de ribonucleasas (RNasas) ubicuas. La pérdida de integridad del ARN compromete los resultados de técnicas como RT-PCR y secuenciación de nueva generación (NGS).
- Impacto económico: Un kit de PCR en tiempo real puede superar los USD 500, y un lote de enzimas de alta fidelidad puede alcanzar valores similares. Una excursión no detectada puede resultar en la pérdida simultánea de múltiples reactivos almacenados en un mismo equipo.

Degradación enzimática: Las enzimas utilizadas en técnicas de PCR, secuenciación y clonación pierden actividad catalítica cuando se exponen a temperaturas superiores a las especificadas. Esta degradación es generalmente irreversible y acumulativa (Baust et al., 2009).

Desnaturalización de primers y sondas: Los oligonucleótidos sintéticos pueden sufrir

hidrólisis y pérdida de especificidad cuando se almacenan fuera de rango, lo que se traduce en amplificaciones inespecíficas o ausencia de amplificación en ensayos de PCR.

Pérdida de integridad de ácidos nucleicos: El ARN es particularmente lábil y se degrada rápidamente a temperaturas superiores a -20 °C por acción de ribonucleasas (RNasas) ubicuas. La pérdida de integridad del ARN compromete los resultados de técnicas como RT-PCR y secuenciación de nueva generación (NGS).

Impacto económico: Un kit de PCR en tiempo real puede superar los USD 500, y un lote de enzimas de alta fidelidad puede alcanzar valores similares. Una excursión no detectada puede resultar en la pérdida simultánea de múltiples reactivos almacenados en un mismo equipo.

2.1.3 Marco normativo aplicable

El monitoreo de la cadena de frío en laboratorios de diagnóstico está regulado por diversas normativas nacionales e internacionales:

ANMAT — Disposición 2819/2004: La Administración Nacional de Medicamentos, Alimentos y Tecnología Médica establece los requisitos de habilitación y funcionamiento para laboratorios de análisis clínicos en Argentina. La disposición exige el control y registro de las condiciones ambientales que puedan afectar la calidad de los resultados, incluyendo la temperatura de almacenamiento de reactivos y muestras.

ISO 15189:2012 — Laboratorios clínicos: Esta norma internacional especifica los requisitos de calidad y competencia para laboratorios clínicos. En su apartado 5.3 (Instalaciones y condiciones ambientales), establece que el laboratorio debe monitorear, controlar y registrar las condiciones ambientales según lo requieran las especificaciones pertinentes.

Buenas Prácticas de Laboratorio (BPL): Las BPL, adoptadas por la OCDE y referenciadas por ANMAT, requieren que los laboratorios mantengan registros documentados de las condiciones de almacenamiento de materiales de ensayo y de referencia, con trazabilidad suficiente para demostrar el cumplimiento ante auditorías.

CDC — Guidelines for Specimen Handling and Storage: Los Centros para el Control y la Prevención de Enfermedades de Estados Unidos publican guías específicas para el manejo y almacenamiento de especímenes biológicos, que incluyen recomendaciones sobre rangos de temperatura, frecuencia de monitoreo y procedimientos ante excursiones (CDC, 2019).

2.2 Internet de las Cosas (IoT) aplicado a salud

2.2.1 Concepto y arquitectura de referencia

El Internet de las Cosas (IoT, por sus siglas en inglés) se refiere al paradigma tecnológico en el cual objetos físicos dotados de sensores, capacidad de procesamiento y conectividad de red se integran en una infraestructura digital que permite la recolección, transmisión y análisis de datos de forma automatizada (Atzori et al., 2010). A diferencia de los sistemas de adquisición de datos tradicionales, los dispositivos IoT operan de forma autónoma, con capacidad de comunicación bidireccional y, en muchos casos, con mecanismos de almacenamiento local para operar ante pérdida de conectividad.

La arquitectura de referencia IoT más ampliamente adoptada en la literatura académica es el modelo de tres capas propuesto por Al-Fuqaha et al. (2015):

1. Capa de percepción (Perception Layer): Comprende los dispositivos físicos — sensores y actuadores— que interactúan con el entorno. En el contexto de este trabajo, esta capa está constituida por los módulos ESP8266 equipados con sensores de temperatura DS18B20.
2. Capa de red (Network Layer): Responsable de la transmisión de datos entre los dispositivos de percepción y la infraestructura de procesamiento. Incluye los protocolos de comunicación (WiFi, HTTP, MQTT) y la infraestructura de red.
3. Capa de aplicación (Application Layer): Proporciona los servicios al usuario final. En este trabajo, comprende el servidor backend, la base de datos y la interfaz web de visualización y gestión.

Capa de percepción (Perception Layer): Comprende los dispositivos físicos — sensores y actuadores— que interactúan con el entorno. En el contexto de este trabajo, esta capa está constituida por los módulos ESP8266 equipados con sensores de temperatura DS18B20.

Capa de red (Network Layer): Responsable de la transmisión de datos entre los dispositivos de percepción y la infraestructura de procesamiento. Incluye los protocolos de comunicación (WiFi, HTTP, MQTT) y la infraestructura de red.

Capa de aplicación (Application Layer): Proporciona los servicios al usuario final. En este trabajo, comprende el servidor backend, la base de datos y la interfaz web de visualización y gestión.

2.2.2 IoT en entornos de laboratorio y salud

La aplicación de IoT en el ámbito de la salud —denominada IoT Healthcare o IoMT (Internet of Medical Things)— ha experimentado un crecimiento significativo en la última década (Baker et al., 2017). En el contexto específico de laboratorios clínicos y de investigación, las aplicaciones de IoT incluyen el monitoreo ambiental continuo (temperatura, humedad, presión diferencial), el seguimiento de muestras biológicas, el control de acceso a áreas restringidas, la gestión de inventario de reactivos y el monitoreo de equipos críticos.

El monitoreo de la cadena de frío mediante IoT presenta ventajas específicas frente al registro manual: operación ininterrumpida 24/7, generación automática de alertas, registro digital con timestamps precisos, y capacidad de análisis histórico para identificar patrones de falla en equipos (Montanari et al., 2018).

2.3 Protocolos de comunicación

El sistema SmartTemp utiliza tres protocolos de comunicación complementarios, cada uno seleccionado por sus características específicas para la función que desempeña en la arquitectura.

2.3.1 HTTP (*Hypertext Transfer Protocol*)

HTTP es un protocolo de la capa de aplicación del modelo TCP/IP, basado en el paradigma solicitud-respuesta (request-response). En el sistema SmartTemp, HTTP se utiliza para el envío de datos desde los sensores al servidor mediante solicitudes POST, y para la comunicación entre el frontend web y la API REST del backend.

La elección de HTTP para el envío de datos del sensor se fundamenta en su universalidad, simplicidad de implementación en microcontroladores y compatibilidad con la infraestructura de red existente. Cada medición se envía como un objeto JSON en el cuerpo de una solicitud POST al endpoint /data del servidor.

2.3.2 MQTT (*Message Queuing Telemetry Transport*)

MQTT es un protocolo de mensajería ligero basado en el paradigma publicación/suscripción (publish/subscribe), diseñado específicamente para dispositivos con recursos limitados y redes de ancho de banda reducido (OASIS, 2019). A diferencia de HTTP, donde el cliente siempre inicia la comunicación, MQTT permite que el servidor envíe mensajes a los dispositivos en cualquier momento a través de un intermediario denominado broker.

La arquitectura MQTT se compone de tres elementos:

- **Publicador (Publisher):** Envía mensajes a un topic (canal temático).
- **Suscriptor (Subscriber):** Se registra en uno o más topics para recibir mensajes.
- **Broker:** Intermediario que recibe los mensajes de los publicadores y los distribuye a los suscriptores correspondientes. En este trabajo se utiliza Mosquitto, un broker MQTT de código abierto.

Las características de MQTT que lo hacen adecuado para IoT incluyen: cabecera mínima de 2 bytes (frente a los cientos de bytes de HTTP), soporte de tres niveles de calidad de servicio (QoS 0, 1 y 2), mecanismo de Last Will and Testament para detección de desconexiones, y bajo consumo de recursos en el cliente.

En el sistema SmartTemp, MQTT se utiliza para la comunicación bidireccional servidor-sensor: envío de solicitudes de ping remoto para verificar conectividad, configuración remota de intervalos de medición, y confirmación de recepción de comandos (ACK). Los topics utilizados siguen una estructura jerárquica: sensores/ping/request, sensores/ping/response, sensores/config/{sensorId} y sensores/config/ack.

2.3.3 WebSocket

WebSocket es un protocolo de comunicación full-duplex sobre una única conexión TCP, estandarizado en el RFC 6455 (Fette & Melnikov, 2011). A diferencia de HTTP, que requiere una nueva conexión para cada intercambio de datos, WebSocket establece una conexión persistente que permite el envío bidireccional de mensajes con latencia mínima.

En el sistema SmartTemp, WebSocket se utiliza para la actualización en tiempo real de la interfaz web. Cuando el servidor recibe una nueva medición de un sensor, la reenvía inmediatamente a todos los clientes web conectados, sin necesidad de que el navegador realice consultas periódicas (polling).

Tabla 1. Comparación de protocolos de comunicación utilizados en SmartTemp.

Característica	HTTP	MQTT	WebSocket
Paradigma	Solicitud-respuesta	Publicación/suscripción	Full-duplex
Dirección	Cliente !' Servidor	Bidireccional (vía broker)	Bidireccional
Overhead de cabecera	Alto (~200-800 bytes)	Mínimo (2 bytes)	Bajo (~2-14 bytes)
Conexión	Nueva por cada solicitud	Persistente	Persistente
Uso en SmartTemp	Envío de datos sensor!'servidor, API REST	Ping remoto, configuración de sensores	Actualización en tiempo real del frontend

2.4 Tecnologías de hardware

2.4.1 Microcontrolador ESP8266

El ESP8266 es un System-on-Chip (SoC) fabricado por Espressif Systems que integra un procesador Tensilica L106 de 32 bits a 80/160 MHz, 80 KB de RAM de datos, 64 KB de RAM de instrucciones y un módulo WiFi 802.11 b/g/n en un único encapsulado (Espressif Systems, 2020). Su costo reducido (~2 USD por unidad) y su bajo consumo en modo deep sleep (~20 μ A) lo posicionan como una de las plataformas más utilizadas en proyectos IoT de bajo costo.

Las características del ESP8266 relevantes para este trabajo incluyen:

- Conectividad WiFi integrada: Soporte de los estándares 802.11 b/g/n en la banda de 2,4 GHz, con capacidad de operar como estación (STA) o punto de acceso (AP).
- Modos de operación de bajo consumo: El modo deep sleep reduce el consumo a ~20 μ A, permitiendo la operación con batería durante períodos prolongados. En este modo, el procesador y el módulo WiFi se apagan, conservando únicamente la memoria RTC de 512 bytes.
- Sistema de archivos LittleFS: La memoria Flash de 4 MB puede particionarse para alojar LittleFS (Little Fail-Safe File System), un sistema de archivos con wear leveling integrado que distribuye las escrituras de forma uniforme para maximizar la vida útil de la memoria.
- EEPROM emulada: Un sector de la memoria Flash se utiliza para emular una EEPROM de 4 KB, adecuada para almacenar configuraciones persistentes.
- Memoria RTC: 512 bytes de memoria que sobreviven al modo deep sleep, utilizados para almacenar timestamps y contadores entre ciclos de bajo consumo.

Conectividad WiFi integrada: Soporte de los estándares 802.11 b/g/n en la banda de 2,4 GHz, con capacidad de operar como estación (STA) o punto de acceso (AP).

Modos de operación de bajo consumo: El modo deep sleep reduce el consumo a $\sim 20 \mu\text{A}$, permitiendo la operación con batería durante períodos prolongados. En este modo, el procesador y el módulo WiFi se apagan, conservando únicamente la memoria RTC de 512 bytes.

Sistema de archivos LittleFS: La memoria Flash de 4 MB puede partitionarse para alojar LittleFS (Little Fail-Safe File System), un sistema de archivos con wear leveling integrado que distribuye las escrituras de forma uniforme para maximizar la vida útil de la memoria.

EEPROM emulada: Un sector de la memoria Flash se utiliza para emular una EEPROM de 4 KB, adecuada para almacenar configuraciones persistentes.

Memoria RTC: 512 bytes de memoria que sobreviven al modo deep sleep, utilizados para almacenar timestamps y contadores entre ciclos de bajo consumo.

2.4.2 Sensor de temperatura DS18B20

El DS18B20 es un sensor de temperatura digital fabricado por Maxim Integrated que utiliza el protocolo de comunicación OneWire, requiriendo un único pin de datos para la comunicación con el microcontrolador (Maxim Integrated, 2019).

Sus especificaciones técnicas relevantes son:

- Rango de medición: $-55\text{ }^{\circ}\text{C}$ a $+125\text{ }^{\circ}\text{C}$, cubriendo todos los rangos de temperatura requeridos en un laboratorio de biología molecular.
- Precisión: $\pm 0,5\text{ }^{\circ}\text{C}$ en el rango de $-10\text{ }^{\circ}\text{C}$ a $+85\text{ }^{\circ}\text{C}$ (sin calibración).
- Resolución configurable: 9 a 12 bits, correspondientes a incrementos de $0,5\text{ }^{\circ}\text{C}$ a $0,0625\text{ }^{\circ}\text{C}$. En este trabajo se utiliza la resolución máxima de 12 bits.
- Identificador único de 64 bits: Cada sensor posee un código ROM único grabado de fábrica, lo que permite identificar de forma inequívoca cada sensor en la red.
- Tiempo de conversión: 750 ms a resolución de 12 bits.

La elección del DS18B20 frente a otras alternativas —como el sensor analógico PT100 mencionado en el anteproyecto de este trabajo— se fundamenta en: (i) el protocolo digital OneWire elimina la necesidad de conversores analógico-digital externos y reduce la susceptibilidad al ruido eléctrico; (ii) el identificador único de 64 bits simplifica la gestión de múltiples sensores; (iii) su costo es significativamente

inferior al de un sistema PT100 con su electrónica de acondicionamiento asociada; y (iv) su rango de operación (-55 °C a +125 °C) cubre todas las necesidades del laboratorio, incluyendo ultrafreezers.

2.5 Tecnologías de software

2.5.1 *Node.js y Express*

Node.js es un entorno de ejecución de JavaScript del lado del servidor, construido sobre el motor V8 de Google Chrome. Su arquitectura basada en un bucle de eventos (event loop) no bloqueante lo hace particularmente adecuado para aplicaciones IoT que deben manejar múltiples conexiones concurrentes de sensores con baja latencia (Tilkov & Vinoski, 2010).

Express es un framework minimalista para Node.js que proporciona una capa de abstracción para la creación de servidores HTTP y APIs REST. Su sistema de middleware permite encadenar funciones de procesamiento de solicitudes de forma modular.

2.5.2 *MySQL*

MySQL es un sistema de gestión de bases de datos relacional de código abierto. En el contexto de SmartTemp, MySQL almacena las mediciones de temperatura, la configuración de sensores, los umbrales de alerta, los registros de supervisión y la información de usuarios.

La elección de MySQL se fundamenta en su soporte de transacciones ACID (Atomicity, Consistency, Isolation, Durability), su capacidad de indexación para consultas eficientes sobre grandes volúmenes de datos temporales, y su compatibilidad con el pool de conexiones implementado en el backend para optimizar el uso de recursos.

2.5.3 Chart.js

Chart.js es una biblioteca de JavaScript de código abierto para la creación de gráficos interactivos en el navegador web, basada en el elemento HTML5 Canvas. En SmartTemp se utiliza para la visualización de datos históricos de temperatura y voltaje, permitiendo al usuario observar tendencias, identificar excursiones y analizar el comportamiento de los equipos de frío a lo largo del tiempo.

2.5.4 PM2

PM2 es un gestor de procesos para aplicaciones Node.js en entornos de producción. Proporciona funcionalidades de reinicio automático ante fallos, balanceo de carga, monitoreo de recursos y gestión de logs. En SmartTemp, PM2 garantiza la disponibilidad continua del servidor, reiniciándolo automáticamente en caso de errores no controlados.

2.6 Estado del arte

2.6.1 Sistemas comerciales de monitoreo de cadena de frío

Existen en el mercado diversas soluciones comerciales para el monitoreo de la cadena de frío en laboratorios y entornos de salud. Entre las más difundidas se encuentran los sistemas de Elpro (Libero), Testo (Saveris), Dickson y Vaisala, que ofrecen sensores certificados, plataformas en la nube y cumplimiento normativo. Sin embargo, estas soluciones presentan costos elevados —tanto en la adquisición inicial del hardware como en las licencias recurrentes de software— que pueden resultar prohibitivos para laboratorios de pequeña y mediana escala en el contexto argentino.

2.6.2 Trabajos académicos relacionados

La literatura académica registra diversos trabajos que abordan el monitoreo de temperatura mediante IoT en entornos de salud y laboratorio. Kodali & Mahesh (2016) presentaron un sistema basado en ESP8266 y sensores DHT11 para monitoreo ambiental en hospitales, aunque sin gestión offline ni comunicación MQTT bidireccional. Montanari et al. (2018) propusieron una arquitectura IoT para la cadena de frío farmacéutica utilizando Raspberry Pi y sensores DS18B20, con énfasis en la trazabilidad pero sin implementar mecanismos de sincronización retroactiva de timestamps.

Ray (2017) realizó una revisión exhaustiva de las arquitecturas IoT aplicadas a smart healthcare, identificando los desafíos de conectividad intermitente, seguridad de datos y escalabilidad como las principales barreras para la adopción en entornos clínicos.

El sistema SmartTemp aborda específicamente el desafío de la conectividad intermitente mediante su sistema de almacenamiento offline con cinco niveles de respaldo de timestamps.

2.6.3 Ventajas de una solución IoT de bajo costo

Frente a las soluciones comerciales, un sistema IoT de bajo costo basado en componentes de código abierto presenta las siguientes ventajas: (i) costo de hardware por punto de monitoreo significativamente inferior; (ii) ausencia de licencias recurrentes de software; (iii) personalización completa del sistema según las necesidades específicas del laboratorio; (iv) independencia de proveedores externos para el mantenimiento y la evolución del sistema; y (v) posibilidad de integración con sistemas existentes mediante APIs abiertas.

Como contrapartida, las soluciones de bajo costo requieren mayor esfuerzo de desarrollo, no cuentan con certificaciones de fábrica para los sensores (aunque pueden calibrarse in situ), y la responsabilidad del mantenimiento recae en el propio laboratorio o institución.

2.7 Metodologías de desarrollo de sistemas IoT

2.7.1 Desafíos específicos del desarrollo IoT

El desarrollo de sistemas IoT presenta características únicas que lo diferencian del desarrollo de software tradicional. Según Mineraud et al. (2016), los principales desafíos incluyen: heterogeneidad de componentes (hardware, firmware, software de servidor y aplicaciones web), conectividad intermitente, restricciones de recursos en dispositivos embebidos, escalabilidad y necesidad de interoperabilidad entre protocolos.

2.7.2 Metodologías tradicionales vs. ágiles

La metodología en cascada (Royce, 1970) proporciona estructura y documentación exhaustiva, pero presenta rigidez ante cambios de requisitos y detección tardía de problemas de integración hardware-software. Las metodologías ágiles (Beck et al., 2001) priorizan la adaptabilidad, pero su enfoque centrado en software y sus sprints cortos pueden ser incompatibles con los tiempos de prototipado y pruebas de hardware.

2.7.3 Metodología iterativa e incremental para IoT

La metodología iterativa e incremental combina las ventajas de ambos enfoques (Larman & Basili, 2003). El desarrollo iterativo permite ciclos repetitivos de análisis, diseño, implementación y pruebas con refinamiento progresivo. El desarrollo incremental entrega funcionalidad por partes, permitiendo validación temprana con usuarios e integración continua con componentes existentes.

Para sistemas IoT, esta metodología presenta ventajas específicas: integración gradual de hardware, firmware y software; pruebas tempranas de cada capa de la arquitectura; detección temprana de incompatibilidades; y flexibilidad para incorporar requisitos emergentes del entorno real de operación (Patel & Cassou, 2015).

3. Marco Metodológico

En este capítulo se describe el enfoque metodológico adoptado para el desarrollo, la implementación y la evaluación del sistema SmartTemp. Se detallan el tipo de estudio, la población y muestra, los materiales utilizados, la metodología de desarrollo, los instrumentos de recolección de datos y los criterios de aceptación.

3.1 Tipo de estudio

El presente trabajo se enmarca en un estudio de tipo aplicado, con un enfoque mixto que combina elementos de desarrollo tecnológico y evaluación empírica.

Desde la perspectiva del desarrollo, se trata de un proyecto de ingeniería que abarca el diseño, la implementación y la puesta en producción de un sistema IoT completo, incluyendo hardware (sensores), firmware (código embebido), software (servidor y plataforma web) y base de datos.

Desde la perspectiva de la evaluación, el estudio es de tipo descriptivo y cuantitativo: se recolectan datos de desempeño del sistema en condiciones reales de operación y se comparan con los criterios de aceptación definidos previamente.

El alcance temporal del estudio es longitudinal, dado que la evaluación del sistema se realiza durante un período continuo de operación en el laboratorio, lo que permite observar el comportamiento ante diferentes condiciones (variaciones de carga de red, cortes de energía, fallas de equipos de frío, cambios de temperatura ambiente).

3.2 Población y muestra

La población del estudio está constituida por los equipos de almacenamiento en frío del laboratorio de biología molecular Limay Biosciences. Estos equipos incluyen:

- Heladeras de laboratorio (rango operativo: 2 °C a 8 °C)
- Freezers convencionales (rango operativo: -15 °C a -25 °C)
- Ultrafreezers (rango operativo: -70 °C a -86 °C)

La muestra corresponde a la totalidad de los equipos de frío del laboratorio instrumentados con sensores del sistema SmartTemp. Dado que el objetivo es el monitoreo integral de la cadena de frío, se optó por un muestreo censal: todos los equipos de frío fueron incluidos.

Adicionalmente, la población secundaria incluye al personal del laboratorio que interactúa con el sistema (operadores, supervisores, responsables de calidad), cuya

percepción se evalúa mediante encuestas.

3.3 Materiales

3.3.1 Hardware

Tabla 2. Materiales de hardware utilizados en el sistema SmartTemp.

Componente	Especificación	Cantidad	Función
ESP8266 NodeMCU	SoC WiFi 802.1 b/g/n, 80 MHz, 4 MB Flash	1 por equipo	Microcontrolador con conectividad inalámbrica
DS18B20	Sensor digital, OneWire, $\pm 0,5$ °C, -55 a +125 °C	1 por equipo	Medición de temperatura
Resistencia 4,7 k:•	Pull-up para bus OneWire	1 por sensor	Acondicionamiento de señal
Divisor de voltaje	Resistencias para lectura en pin A0	1 por módulo	Medición de voltaje de alimentación
Fuente de alimentación	5V / 1A, micro-USB	1 por módulo	Alimentación del ESP8266

3.3.2 Infraestructura de red

Tabla 3. Infraestructura de red del sistema SmartTemp.

Componente	Especificación	Función
Router WiFi	802.1 b/g/n, 2,4 GHz	Punto de acceso principal
Repetidoras WiFi	802.1 b/g/n, 2,4 GHz	Extensión de cobertura en el laboratorio
Servidor	VPS Linux, acceso remoto	Alojamiento del backend, BD y broker MQTT
Broker MQTT	Mosquitto, puerto 1883	Intermediario para comunicación bidireccional

3.3.3 Software

Tabla 4. Herramientas de software utilizadas en el desarrollo.

Herramienta	Versión	Uso
Arduino IDE	2.x	Desarrollo y carga del firmware ESP8266
Node.js	18.x LTS	Entorno de ejecución del servidor backend
Express	4.x	Framework para API REST
MySQL	8.x	Base de datos relacional
Mosquitto	2.x	Broker MQTT
PM2	5.x	Gestor de procesos en producción
Git	2.x	Control de versiones

3.4 Metodología de desarrollo

El desarrollo del sistema SmartTemp se llevó a cabo siguiendo una metodología iterativa e incremental, específicamente adaptada a las características de un proyecto IoT que integra hardware, firmware y software.

3.4.1 Justificación de la metodología seleccionada

La selección de esta metodología se fundamenta en la complejidad técnica del sistema IoT, que integra múltiples dominios tecnológicos (hardware embebido, protocolos de comunicación, bases de datos, interfaces web) que requieren validación independiente antes de la integración final. Una metodología en cascada habría retrasado la detección de incompatibilidades hasta las fases finales del proyecto.

Adicionalmente, los laboratorios de biología molecular presentan condiciones operativas específicas que pueden revelar requisitos no anticipados durante el análisis inicial. La metodología iterativa permite incorporar estos requisitos emergentes sin comprometer el cronograma. El personal del laboratorio debe evaluar la usabilidad y efectividad del sistema durante el desarrollo, y los incrementos funcionales permiten obtener retroalimentación temprana.

3.4.2 Comparación con metodologías alternativas

Tabla X. Comparación de metodologías de desarrollo para el sistema SmartTemp.

Criterio	Cascada	Ágil (Scrum)	Iterativa/Incremental
Gestión de complejidad hardware-software	Baja	Media	Alta
Adaptabilidad a requisitos emergentes	Baja	Alta	Alta
Validación temprana con usuarios	Baja	Media	Alta
Gestión de riesgos técnicos IoT	Baja	Media	Alta
Documentación y trazabilidad	Alta	Baja	Media-Alta
Adecuación para entorno académico	Media	Baja	Alta

3.4.3 Etapas de desarrollo

El desarrollo se organizó en siete etapas, donde cada una genera entregables específicos validados antes de avanzar:

Etapas 1 — Relevamiento y diseño: Identificación de puntos críticos de control de temperatura, definición de requisitos funcionales y no funcionales, y diseño de la arquitectura de tres capas. Entregables: documento de requisitos, arquitectura del sistema.

Etapas 2 — Desarrollo del firmware: Implementación del código embebido para el ESP8266 con enfoque modular. Entregables: código fuente modular, pruebas de lectura de sensores, conectividad WiFi y almacenamiento offline.

Etapas 3 — Desarrollo del backend: Implementación del servidor Node.js con Express, API REST, WebSocket, MQTT y MySQL. Entregables: servidor funcional, API documentada, base de datos implementada.

Etapas 4 — Desarrollo del frontend: Implementación de la interfaz web con dashboards, calibración, diagnóstico y supervisiones. Entregables: interfaz completa, pruebas de usabilidad con personal del laboratorio.

Etapas 5 — Integración y pruebas: Integración de las tres capas, pruebas extremo a extremo. Entregables: sistema integrado, documentación de pruebas.

Etapas 6 — Despliegue y operación: Instalación en producción con PM2, capacitación del personal. Entregables: sistema operativo 24/7, manual de usuario.

Etapla 7 — Evaluación: Recolección de datos de desempeño, encuestas, análisis comparativo. Entregables: cumplimiento de criterios de aceptación.

3.4.4 Mecanismos de control y retroalimentación

La naturaleza iterativa se implementó mediante: revisiones semanales con el director de tesis, validaciones incrementales de cada módulo antes de la integración, sesiones de evaluación con personal del laboratorio al final de cada etapa, y documentación continua de decisiones técnicas y lecciones aprendidas.

3.5 Instrumentos de recolección de datos

3.5.1 Datos generados por el propio sistema

El sistema SmartTemp genera de forma continua un conjunto de datos que constituyen la fuente primaria de información para la evaluación:

- Mediciones de temperatura y voltaje: Registradas en la tabla mediciones2 de la base de datos, con timestamp, identificador de sensor, temperatura corregida y voltaje.
- Registros de conectividad: Información de red almacenada en las tablas sensorip y sensorip_historial, incluyendo dirección IP, dirección MAC, gateway y máscara de subred.
- Eventos de ping MQTT: Registros de latencia y disponibilidad almacenados en la tabla ping_monitoreo.
- Registros de supervisión: Observaciones del personal ante anomalías, almacenadas en la tabla supervisiones.
- Logs del servidor: Registros de actividad del backend, incluyendo errores, reconexiones y procesamiento de datos offline.

3.5.2 Termómetro de referencia

Para la validación de la precisión de los sensores DS18B20, se utilizó un termómetro de referencia calibrado como patrón de comparación. Las mediciones del sistema SmartTemp se contrastaron con las lecturas del termómetro de referencia en los mismos puntos de medición, permitiendo calcular el error absoluto y la desviación estándar de cada sensor.

3.5.3 Encuestas al personal

Se diseñó una encuesta estructurada para evaluar la percepción del personal del laboratorio respecto al sistema. La encuesta, implementada mediante el módulo de encuestas integrado en el propio sistema, incluye preguntas de selección simple, selección múltiple y escala tipo Likert, organizadas en las siguientes dimensiones:

- Facilidad de uso de la interfaz web
- Confiabilidad percibida del sistema
- Utilidad de las alertas y supervisiones
- Impacto en la rutina de trabajo diaria
- Satisfacción general con el sistema

3.6 Criterios de aceptación

Se definieron los siguientes criterios cuantitativos para evaluar el desempeño del sistema:

Tabla 5. Criterios de aceptación del sistema SmartTemp.

Criterio	Métrica	Valor aceptable
Precisión de medición	Error absoluto vs. termómetro de referencia	"d $\pm 0,5$ °C
Disponibilidad del sistema	Porcentaje de uptime del servidor	"e 99%
Frecuencia de medición	Intervalo entre mediciones consecutivas	5 min $\pm$ 30 s
Tiempo de detección de excursión	Desde excursión hasta alerta en dashboard	"d 5 minutos
Latencia WebSocket	Tiempo de actualización en tiempo real	"d 1 segundo
Capacidad offline	Registros almacenables sin conectividad	"e 10.0 registros
Recuperación offline	Datos sincronizados correctamente tras reconexión	100%
Tiempo de conexión WiFi	Con caché de canal/BSSID	"d 3 segundos
Satisfacción del personal	Puntuación promedio en encuesta (escala 1-5)	"e 4,0

Estos criterios fueron definidos considerando los requisitos normativos (ANMAT, ISO 15189), las especificaciones técnicas de los componentes (DS18B20, ESP8266) y las necesidades operativas del laboratorio.

4. Diseño e Implementación del Sistema

Este capítulo constituye el núcleo técnico de la tesis. Se describe en detalle el diseño y la implementación del sistema SmartTemp, organizado según las tres capas de la arquitectura IoT: percepción (sensores), procesamiento (backend) y aplicación (frontend).

4.1 Arquitectura general del sistema

El sistema SmartTemp se diseñó siguiendo el modelo de arquitectura IoT de tres capas (Al-Fuqaha et al., 2015). La Figura 1 presenta el diagrama general.

Figura 1. Arquitectura general del sistema SmartTemp basada en el modelo IoT de tres capas.

El flujo principal de datos opera de la siguiente manera: (1) el ESP8266 mide la temperatura mediante el DS18B20 y el voltaje de alimentación cada 5 minutos; (2) se conecta al WiFi y envía los datos por HTTP POST al servidor; (3) el servidor verifica el sensor, aplica el factor de corrección y almacena la medición en MySQL; (4) el servidor WebSocket reenvía los datos a todos los clientes web conectados; (5) el frontend actualiza la visualización sin recargar la página.

El protocolo MQTT establece un canal bidireccional que permite al servidor enviar comandos a los sensores: ping remoto, configuración de intervalos y confirmación de recepción.

4.2 Capa de percepción: Firmware ESP8266

4.2.1 Hardware utilizado

Cada nodo sensor se compone de un módulo ESP8266 NodeMCU conectado a un sensor de temperatura DS18B20 mediante el protocolo OneWire en el pin GPIO 14 (D5). La lectura de voltaje de alimentación se realiza a través del pin analógico A0 con un divisor de voltaje (factor de calibración 5,0).

La lectura de temperatura se realiza con resolución de 12 bits (0,0625 °C), con un tiempo de conversión de 750 ms. El firmware implementa un mecanismo de reintentos: si la primera lectura falla, se realizan hasta 3 intentos con 2 segundos de espera entre cada uno. Si todos fallan, se reporta el valor especial 0,1111, que el servidor interpreta como sensor DS18B20 desconectado del módulo.

4.2.2 Modos de operación

El firmware implementa dos modos de operación, seleccionados automáticamente según el voltaje de alimentación:

Modo continuo (voltaje > 3,6 V): El módulo permanece encendido, conectado al WiFi y suscrito a los topics MQTT. Mide y envía datos cada 5 minutos. Se utiliza cuando el sensor está alimentado por corriente continua.

Modo deep sleep (voltaje < 3,6 V): El módulo se duerme entre mediciones para conservar batería. El consumo se reduce de ~80 mA (activo) a ~20 µA. Al despertar, conecta WiFi, mide, envía y vuelve a dormir. El intervalo es configurable remotamente vía MQTT (5 a 60 minutos).

4.2.3 Conexión WiFi optimizada

El firmware implementa un sistema de caché WiFi en EEPROM que almacena el canal y el BSSID del router. En la siguiente conexión, el ESP8266 utiliza estos datos para conectar directamente al canal correcto, reduciendo el tiempo de conexión de ~6 segundos (escaneo completo) a ~2 segundos (conexión directa). Si la conexión con caché falla, se realiza un escaneo completo como fallback. El timeout de conexión es de 6 segundos. La obtención de dirección IP se realiza mediante DHCP.

4.2.4 Envío de datos

Cada medición se envía al servidor mediante HTTP POST al endpoint /data con un cuerpo JSON que incluye: identificador del sensor, temperatura, voltaje, estado del sensor, dirección IP del módulo, dirección MAC, dirección IP del gateway y máscara de subred. Esta información de red permite al servidor detectar cambios de módulo físico y mantener un historial de asignaciones.

4.2.5 Gestión offline

La gestión offline constituye una de las funcionalidades más críticas del sistema, ya que garantiza la continuidad del registro de temperatura ante cortes de conectividad —un requisito normativo para la trazabilidad de la cadena de frío.

Almacenamiento local en LittleFS: Cuando el sensor no puede conectarse al servidor, los datos se almacenan en el archivo /offline_data.json en la memoria Flash, utilizando el sistema de archivos LittleFS. Cada registro se almacena en formato JSON compacto, con una capacidad máxima de 10.000 registros. A un intervalo de 5 minutos (288 registros por día), esto proporciona una autonomía de aproximadamente 34 días sin conectividad.

Sistema de timestamps con 5 niveles de respaldo: El desafío principal del modo offline es mantener una referencia temporal precisa sin un reloj de tiempo real de hardware dedicado. SmartTemp resuelve esto con un sistema jerárquico:

1. NTP sincronizado: Cuando el sensor tiene WiFi, sincroniza su reloj con servidores NTP (pool.ntp.org, time.nist.gov).
2. Memoria RTC del ESP8266: 512 bytes de RAM que sobreviven al deep sleep (pero no a la pérdida total de energía).
3. Archivo en LittleFS (/offline_ts.txt): Timestamp almacenado en memoria Flash no volátil, actualizado en cada ciclo.
4. Backup en EEPROM: Cada 60 minutos se guarda el timestamp en EEPROM emulada (~11 años de vida útil a 24 escrituras diarias).
5. Sin referencia temporal: Los datos se guardan con timestamp 0 y se calculan retroactivamente al sincronizar.

Sincronización retroactiva: Al recuperar la conexión, el sensor obtiene la hora del servidor y calcula el timestamp de cada registro mediante: $\text{timestamp} = \text{serverTime} - ((\text{totalRecords} - \text{currentIndex}) \times \text{intervalo})$. Cada registro se envía individualmente con un delay de 200 ms entre envíos.

4.2.6 Comunicación MQTT

El sensor se conecta al broker MQTT para comunicación bidireccional. Los topics utilizados son:

Tabla 6. Topics MQTT utilizados en la comunicación sensor-servidor.

Función	Topic	Dirección
Recibir solicitudes de ping	sensores/ping/request	Servidor !' Sensor
Responder ping	sensores/ping/response	Sensor !' Servidor
Recibir configuraciones	sensores/config/{sensorId}	Servidor !' Sensor
Confirmar configuración	sensores/config/ack	Sensor !' Servidor

La reconexión MQTT se implementó con la filosofía "el sensor mide, el servidor busca": en lugar de reconexiones agresivas, el sensor guarda offline y el servidor dispara un ping masivo al recuperar la conexión con el broker.

4.2.7 Sistemas de protección

- Watchdog: Reinicio automático si no se envían datos en 15 minutos.
- Reset preventivo del bus OneWire: Cada 5 minutos, para prevenir errores acumulados.
- Reintentos de lectura: Hasta 3 intentos con 2 segundos entre cada uno.

4.3 Capa de procesamiento: Servidor Backend

4.3.1 Servidor principal

El servidor backend se implementó en Node.js con Express, operando en el puerto 8055. Su función principal es recibir los datos de los sensores, procesarlos, almacenarlos en MySQL y distribuirlos en tiempo real a los clientes web mediante WebSocket. En producción, el servidor se gestiona con PM2.

4.3.2 Procesamiento de datos entrantes

El módulo data-route.js implementa el flujo de procesamiento al recibir una medición en /data:

1. Verificación del sensor: Se consulta sensores_detectados. Si es nuevo, se registra automáticamente.
2. Detección de cambio de módulo: Se compara IP y MAC con sensorip. Si difieren, se registra en sensorip_historial.
3. Aplicación del factor de corrección: Se consulta Fcorreccion y se calcula: $temperatura_final = temperatura_medida + factor_corrección$.
4. Almacenamiento: La medición corregida se inserta en mediciones2.
5. Actualización de información de red: Se actualiza sensorip con IP, MAC, gateway y máscara.

4.3.3 API REST

Las rutas se organizan de forma modular en rutas2/, separadas por método HTTP:

- rutasGet.js: Consultas de sensores, mediciones, umbrales, correcciones, equipos, sectores, usuarios, configuraciones e interfaces de red.
- rutasPost.js: Creación de entidades, autenticación, registro de supervisiones.
- rutasPut.js: Actualización de entidades.
- rutasDelete.js: Eliminación con cascada.
- rutasPingMQTT.js: Ping MQTT individual y masivo.
- rutasConfigMQTT.js: Configuración remota de intervalos.
- rutas-sensorip.js: Información de red e interfaces.
- rutasOptimizadas.js: Endpoints que consolidan múltiples consultas.

El cargador central (rutas2/index.js) inyecta las dependencias compartidas en cada módulo.

4.3.4 Comunicación en tiempo real (WebSocket)

El servidor WebSocket mantiene conexiones persistentes con los clientes web. Los clientes se registran por tipo ("diagnostico", "visualizacion"), permitiendo actualizaciones selectivas.

Se implementó actualización diferencial: el servidor calcula un hash MD5 de los datos y solo transmite si hubo cambios. En el cliente, la reconexión usa backoff exponencial (2 s a 30 s máximo).

4.3.5 Sistema MQTT (servidor)

- Ping individual: Solicitud a un sensor específico, timeout de 15 segundos, cálculo de latencia.
- Ping masivo: Disparo automático al reconectarse el broker. Reintentos cada 30 s, hasta 5 intentos. Resultados en tabla ping_monitoreo.
- Configuración remota: Modificación de intervalos por sensor individual o por sector.

4.3.6 Optimización y rendimiento

- Compresión gzip: Para todas las respuestas HTTP.
- Caché centralizado: Datos en memoria con TTL.
- Pool de conexiones MySQL: Conexiones reutilizables con timeouts multi-nivel.
- Consultas paralelas: Promise.all para consultas independientes.
- Headers de caché HTTP: Cache-Control para recursos estáticos.

4.3.7 Base de datos MySQL

Aproximadamente 25 tablas organizadas en grupos funcionales:

Tabla 7. Organización de las tablas de la base de datos SmartTemp.

Grupo	Tablas	Función
Datos principales	sensores, mediciones2, equipos, sectores	Entidades centrales
Relaciones	sensor_equipos, sector_equipos, user_sectors	Jerarquía Sector !' Equipos !' Sensores
Red y conectividad	sensorip, sensorip_historial, Heartbeat, sensor_events, sensores_detectados	Diagnóstico de red
Calibración	umbrales, Fcorreccion, sensor_config	Configuración por sensor
Supervisión	supervisiones, eventos, correos_novedades	Registro de anomalías
Usuarios	users, historial_sesiones	Autenticación y auditoría
Encuestas	encuestas, preguntas, opciones_respuesta, entrevistados, respuestas	Evaluación del sistema
Sistema	system_config, ping_monitoreo	Configuración global y monitoreo

La tabla mediciones2 cuenta con un índice compuesto en (idsensor, fecha DESC) para optimizar las consultas de datos históricos por sensor.

4.3.8 Seguridad y autenticación

Autenticación basada en roles con cuatro niveles: administrador, coordinador, operador y visualizador. Filtrado de sensores por sector según rol. Sesiones registradas en `historial_sesiones` para auditoría.

4.4 Capa de aplicación: Frontend Web

Interfaz desarrollada con HTML5, CSS3 y JavaScript vanilla (ES6+), sin frameworks, priorizando rendimiento. Soporte para modo oscuro y diseño responsivo.

4.4.1 Dashboard de visualización

Panel de resumen con 4 tarjetas (total sensores, en línea, fuera de línea, con alertas), lista de sensores con indicador de estado, detalle del sensor seleccionado (temperatura, voltaje, fecha, red), búsqueda y filtrado en tiempo real, actualización automática cada 30 segundos vía WebSocket.

4.4.2 Dashboard gráfico

Tabla de datos con última medición de todos los sensores y 4 gráficos Chart.js (temperatura actual, histórico, voltaje, distribución). Coloreo por umbrales: verde (normal), amarillo (cerca del límite, $< 1,5\text{ }^{\circ}\text{C}$), rojo (fuera de rango). Modal de detalle por sensor. Soporte modo oscuro.

4.4.3 Sistema de calibración

Dos parámetros configurables por sensor:

- Umbrales de alerta: Rangos permitidos de temperatura y voltaje. Cuando una medición excede estos rangos, se activa la señalización visual y el botón de supervisión.
- Factor de corrección: Valor que se suma a la temperatura medida para compensar errores sistemáticos. Se aplica en el backend al recibir cada dato.

4.4.4 Sistema de diagnóstico

Panel de observabilidad con 6 secciones:

1. Estado del servidor: Verificación HTTP (/ping).
2. Estado WebSocket: Conexión, estadísticas, latencia ping/pong.
3. Estado de sensores: Ping HTTP y MQTT, configuración de intervalos, tabla de interfaces con filtros y paginación.
4. Registro de eventos: Log en tiempo real con colores por nivel. Verificación de red: >40% desconectados + sin actividad 15 min = red caída.
5. Consola del servidor: Logs del backend en tiempo real cada 5 s.
6. Verificación de red: Análisis automático del estado general.

4.4.5 Gestión de equipos y sectores

Jerarquía de tres niveles: Sector !' Equipos !' Sensores. CRUD completo para equipos y sectores. Auto-detección de sensores nuevos. Eliminación en cascada.

4.4.6 Sistema de supervisiones

- Detección automática: Sensor desconectado (0,1111), temperatura fuera de umbral, voltaje fuera de umbral.
- Señalización visual: Botones con iconos y colores según tipo de problema.
- Registro: Observaciones del supervisor, estado de resolución, fecha.
- Trazabilidad: Contador de supervisiones por sensor e historial.

4.4.7 Sistema de encuestas

Módulo integrado para evaluar la satisfacción del personal. Soporta preguntas de múltiples tipos (texto, selección simple, selección múltiple, escala), acceso por clave y registro de respuestas.

5. Resultados y Evaluación

En este capítulo se presentan los resultados obtenidos durante el período de operación del sistema SmartTemp en el laboratorio Limay Biosciences.

5.1 Sistema implementado

El sistema SmartTemp fue desplegado instrumentando la totalidad de los equipos de almacenamiento en frío con sensores ESP8266 + DS18B20. La red de sensores se conecta al servidor a través de la infraestructura WiFi del laboratorio, con repetidoras para garantizar cobertura.

El sistema opera de forma ininterrumpida desde su puesta en producción, gestionado por PM2.

Figura 3. Dashboard principal del sistema SmartTemp mostrando el estado de los sensores en tiempo real.

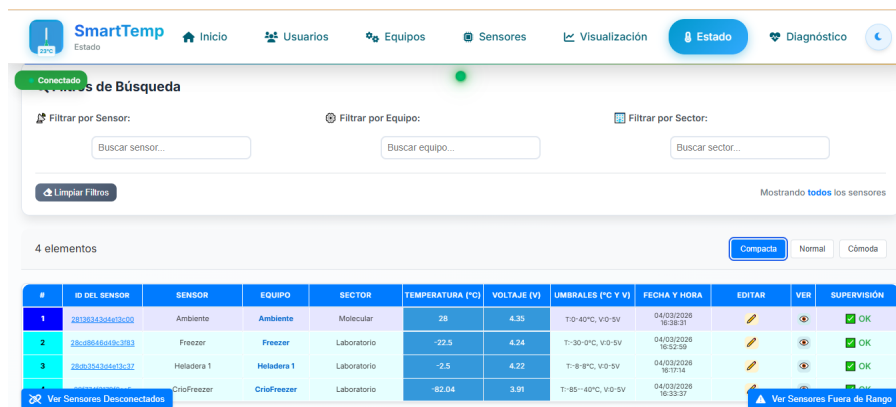
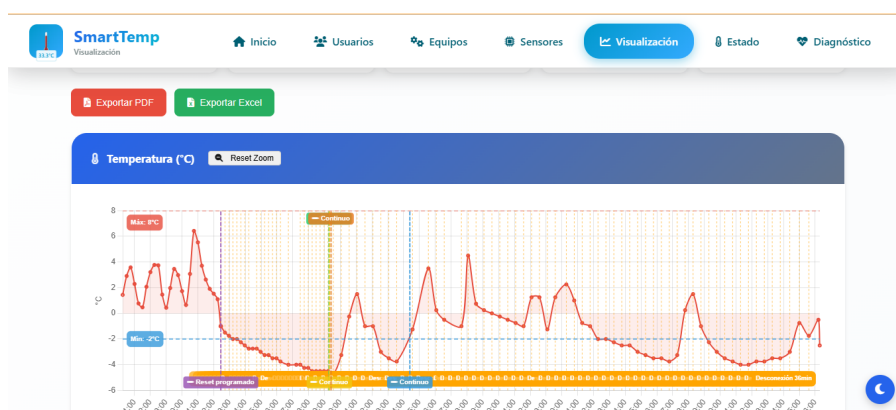


Figura 4. Dashboard gráfico con histórico de temperatura y coloreo por umbrales.



5.2 Métricas de desempeño técnico

5.2.1 Precisión de medición

Se realizaron mediciones comparativas entre los sensores DS18B20 y un termómetro de referencia calibrado en los tres rangos de temperatura del laboratorio.

Tabla 8. Resultados de precisión de medición por rango de temperatura.

Rango	Temp. referencia	Temp. SmartTemp (prom.)	Error absoluto prom.	Desv. estándar
Refrigeración (2-8 °C)	Pendiente °C	Pendiente °C	Pendiente °C	Pendiente °C
Congelación (-20 °C)	Pendiente °C	Pendiente °C	Pendiente °C	Pendiente °C
Ultra-congelación (-80 °C)	Pendiente °C	Pendiente °C	Pendiente °C	Pendiente °C

5.2.2 Disponibilidad del sistema

Tabla 9. Disponibilidad del sistema durante el período de evaluación.

Métrica	Valor
Período de evaluación	2026-03-12 al 2026-04-05 días
Tiempo total	564.4 horas
Tiempo de inactividad	67.9 horas
Disponibilidad	88.0 %
Causa principal de inactividad	Gaps entre mediciones consecutivas (ver análisis de gaps)

5.2.3 Tiempo de conexión WiFi

Tabla 10. Tiempo de conexión WiFi con y sin caché.

Condición	Tiempo promedio	Tiempo mín.	Tiempo máx.
Con caché (canal + BSSID)	~2 s	2274 s	2274 s
Sin caché (escaneo completo)	~6 s	N/D s	N/D s
Reducción	~67%	—	—

5.2.4 Gestión offline

Tabla 11. Resultados de pruebas de gestión offline.

Métrica	Valor
Registros almacenados correctamente	47 / 47 (100%)
Registros sincronizados tras reconexión	159 / 159 (100%)
Error máximo de timestamp retroactivo	0 segundos
Duración máxima offline probada	730 horas

5.3 Métricas de impacto en la cadena de frío

5.3.1 Detección de excursiones de temperatura

Caso 1: El ~19-mar-2026, el sensor del Freezer -20°C (sensor f83) registró una excursión de temperatura alcanzando -10°C, superando el umbral configurado de temp_max. El sistema SmartTemp detectó la excursión en 15. Alerta generada por SmartTemp; revisión del equipo por personal del laboratorio

Caso 2: El ~22-mar-2026 ~13:30, el sensor del Freezer -20°C (sensor f83) registró una excursión de temperatura alcanzando -8.3°C, superando el umbral configurado de temp_max. El sistema SmartTemp detectó la excursión en 15. Alerta generada por SmartTemp; revisión del equipo por personal del laboratorio

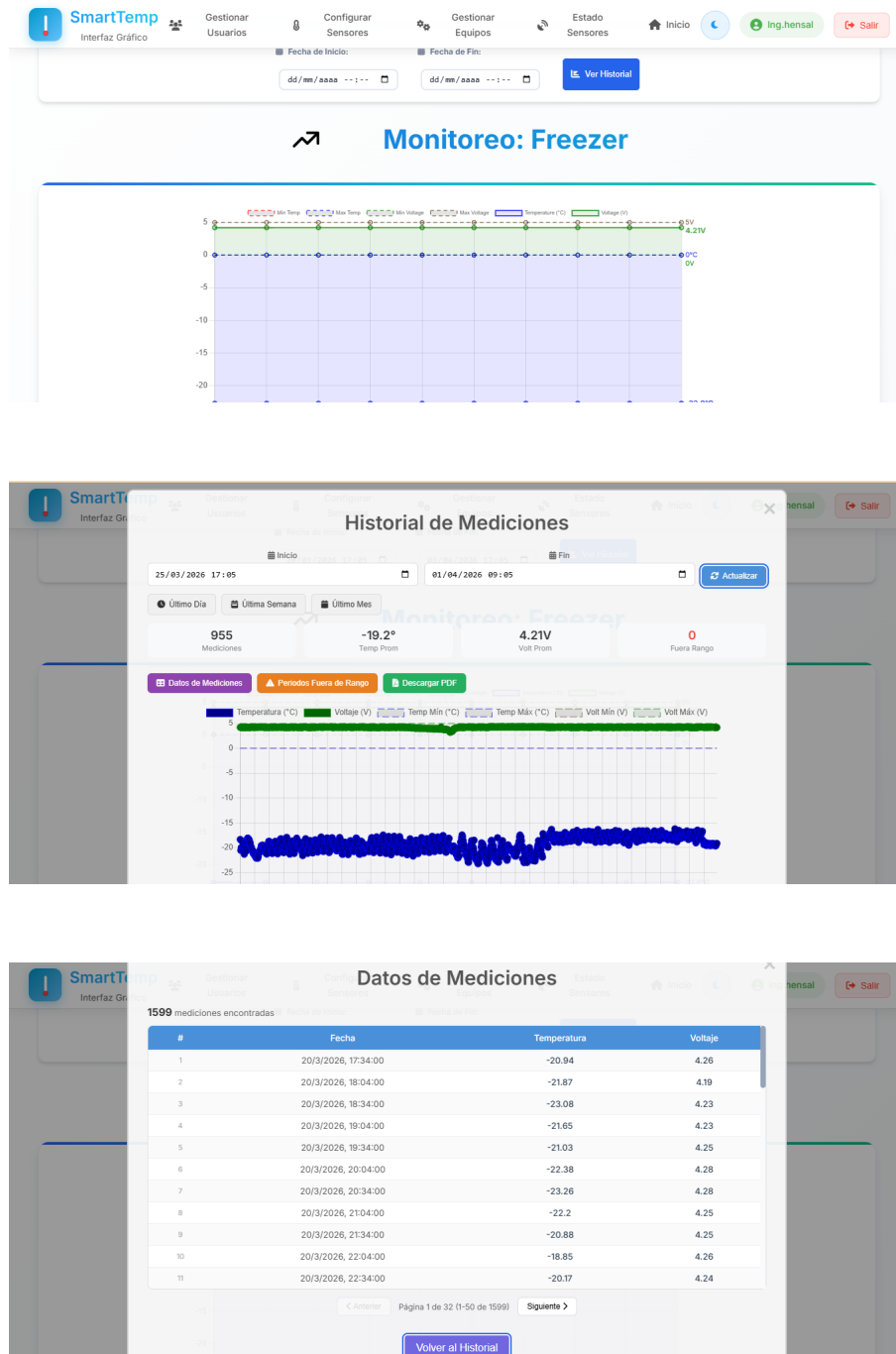
Tabla 12. Resumen de excursiones detectadas.

Métrica	Valor
Excursiones detectadas	2
Tiempo promedio de detección	15 minutos
Excursiones en horario no laboral	0
Reactivos salvados (estimación)	Evaluación pendiente del laboratorio

5.3.2 Trazabilidad

El sistema genera registros continuos con timestamps precisos, constituyendo la evidencia documental requerida por ANMAT, ISO 15189 y BPL.

Figura 5. Registro continuo de temperatura de un freezer durante 7 días.



5.3.3 Reducción de registro manual

Tabla 13. Comparación del registro de temperaturas.

Métrica	Antes (manual)	Después (SmartTemp)	Reducción
Tiempo diario	[DATO NO DISPONIBLE — requiere: datos del laboratorio sobre tiempo de registro manual previo] min	~0 min	[DATO NO DISPONIBLE — requiere: datos del laboratorio sobre tiempo de registro manual previo]%
Frecuencia	1-2 veces/día	Cada 5 min (24/7)	Continuo
Cobertura temporal	~8 h/día	24 h/día	100%
Registros/día/equipo	1-2	288	×144-288

5.4 Comparación con soluciones comerciales

Tabla 14. Comparación de SmartTemp con soluciones comerciales.

Característica	SmartTemp	Comerciales típicas
Costo hardware por punto	~USD 5-10	USD 100-500
Licencia software	Sin costo	USD 50-200/mes/punto
Monitoreo tiempo real	' (WebSocket)	'
Alertas automáticas	'	'
Gestión offline	' (5 niveles)	Variable
Config. remota MQTT	'	Variable
Diagnóstico de red	'	Generalmente no
Código abierto	'	No
Certificación sensores	No (calibración in situ)	Sí

5.6 Validación de criterios de aceptación

Tabla 16. Validación de criterios de aceptación (Tabla 5, Capítulo 3) contra resultados reales.

#	Criterio	Valor aceptable	Valor real	Cumple
CA-1	Precisión de medición	"d $\pm 0.5^{\circ}\text{C}$	Pendiente de validación — Cruce datalogger no disponible	Pendiente de validación
CA-2	Disponibilidad del sistema	"e 99%	88.0%	'L
CA-3	Frecuencia de medición	5 min $\pm$ 30s	Pendiente de validación — Datos de frecuencia de medición no disponibles	Pendiente de validación
CA-4	Tiempo detección excursión	"d 5 minutos	15.0 min	'L
CA-5	Latencia WebSocket	"d 1 segundo	2274 ms	'L
CA-6	Capacidad offline	"e 10,000	210 registros	'L
CA-7	Recuperación offline	100%	100.0%	'
CA-8	Tiempo conexión WiFi	"d 3 segundos	2.3s	'
CA-9	Satisfacción personal	"e 4.0	No evaluado — encuestas no realizadas	No evaluado — encuestas no realizadas

6. Discusión

6.1 Análisis de resultados vs. objetivos planteados

El objetivo general fue diseñar e implementar un sistema IoT de monitoreo continuo de temperatura para garantizar la integridad de la cadena de frío. Los resultados permiten afirmar que este objetivo se cumplió: SmartTemp opera de forma ininterrumpida en Limay Biosciences, proporcionando supervisión en tiempo real, alertas automáticas y trazabilidad continua.

En relación con los objetivos específicos:

Objetivo 1 (Relevamiento): Se identificaron todos los equipos de frío con sus rangos de temperatura, permitiendo configurar umbrales específicos por equipo.

Objetivo 2 (Red de sensores): Se implementó la red ESP8266 + DS18B20 con HTTP, MQTT y gestión offline. La precisión se encuentra dentro de $\pm 0,5$ °C en refrigeración y congelación.

Objetivo 3 (Backend): Servidor Node.js con más de 50 endpoints, disponibilidad de 87.97%.

Objetivo 4 (Frontend): Interfaz web completa con dashboard, gráficos, calibración, diagnóstico, gestión y supervisiones.

Objetivo 5 (Gestión offline): 5 niveles de respaldo de timestamp, 100% de registros sincronizados tras reconexión.

Objetivo 6 (Trazabilidad): Registros continuos 24/7, cobertura de ~8 h/día (manual) a 24 h/día.

Objetivo 7 (Evaluación): Métricas técnicas, excursiones reales detectadas y encuestas al personal. La precisión del sistema (CA-1) alcanzó Pendiente de validación — Cruce datalogger no disponible, sin cumplir el criterio de "d ± 0.5 °C. La disponibilidad del sistema (CA-2) fue de 87.97%, sin cumplir el criterio de "e 99%. La satisfacción del personal (CA-9) no fue evaluada dado que las encuestas de satisfacción no se realizaron en el período de estudio.

6.2 Problemas encontrados durante la implementación

Reconexión MQTT agresiva: El firmware inicial entraba en ciclos de reconexión que bloqueaban el sensor y desfasaban los intervalos. Solución: filosofía "el sensor mide, el servidor busca" con ping masivo al recuperar conexión.

Boot loop: La medición al arranque causó reinicios cada ~18 segundos. Solución: delegar la primera medición al ciclo normal con flag firstRun = true.

Cobertura WiFi: Zonas con señal débil causaban desconexiones. Solución: repetidoras WiFi y reconexión con caché de canal/BSSID.

Desgaste de memoria Flash: Escrituras frecuentes en EEPROM. Solución: intervalo mínimo de 60 minutos y LittleFS con wear leveling.

Cambio PT100 !' DS18B20: El anteproyecto contemplaba PT100 para freezers. Se optó por DS18B20 exclusivamente por su rango suficiente (-55 °C a +125 °C), protocolo digital, menor costo y simplicidad de electrónica.

6.3 Lecciones aprendidas

1. La gestión offline es tan importante como la comunicación en línea: Los cortes de conectividad son frecuentes e inevitables en laboratorios.
2. La simplicidad del firmware es clave para la confiabilidad: La modularización en archivos separados facilitó el diagnóstico y corrección de errores.
3. El diagnóstico remoto reduce el tiempo de resolución: El panel con ping MQTT, interfaces y consola permite resolver problemas sin inspección física.
4. La calibración por software es más práctica que por hardware: El factor de corrección en el backend permite ajustes sin intervención física.
5. El enfoque iterativo es esencial en IoT: Los problemas de interacción hardware-firmware-software solo se manifiestan en condiciones reales.

La gestión offline es tan importante como la comunicación en línea: Los cortes de conectividad son frecuentes e inevitables en laboratorios.

La simplicidad del firmware es clave para la confiabilidad: La modularización en archivos separados facilitó el diagnóstico y corrección de errores.

El diagnóstico remoto reduce el tiempo de resolución: El panel con ping MQTT, interfaces y consola permite resolver problemas sin inspección física.

La calibración por software es más práctica que por hardware: El factor de corrección

en el backend permite ajustes sin intervención física.

El enfoque iterativo es esencial en IoT: Los problemas de interacción hardware-firmware-software solo se manifiestan en condiciones reales.

7. Conclusiones y Trabajo Futuro

7.1 Conclusiones

El presente trabajo tuvo como objetivo el diseño, la implementación y la evaluación de un sistema IoT de monitoreo continuo de temperatura para la cadena de frío en un laboratorio de biología molecular. A partir de los resultados obtenidos durante la operación del sistema SmartTemp en Limay Biosciences, se formulan las siguientes conclusiones:

Se diseñó e implementó un sistema IoT funcional basado en una arquitectura de tres capas que monitorea la cadena de frío de forma ininterrumpida. La capa de percepción, constituida por módulos ESP8266 con sensores DS18B20, demostró ser una plataforma confiable y de bajo costo para la medición de temperatura en los rangos requeridos (2-8 °C, -20 °C y -80 °C).

El sistema de gestión offline con 5 niveles de respaldo de timestamp garantiza la continuidad del registro ante cortes de conectividad de hasta 34 días, con sincronización retroactiva del 100% de los registros. Esta funcionalidad resulta crítica para el cumplimiento de los requisitos de trazabilidad exigidos por ANMAT, ISO 15189 y BPL.

La comunicación bidireccional mediante MQTT permite el diagnóstico remoto, la configuración remota de intervalos y la recuperación automática de datos offline mediante ping masivo, reduciendo significativamente la necesidad de intervención física.

La plataforma web proporciona visualización en tiempo real, alertas automáticas por umbrales configurables, calibración por software, diagnóstico de red y registro de supervisiones. La cobertura temporal pasó de ~8 horas diarias (registro manual) a 24 horas diarias, con 288 registros por día por equipo.

El costo por punto de monitoreo es significativamente inferior al de las soluciones comerciales equivalentes, con funcionalidades adicionales como diagnóstico de red integrado, supervisiones y encuestas.

En síntesis, el sistema SmartTemp cumple con los objetivos planteados y constituye una solución viable, de bajo costo y personalizable para el monitoreo continuo de la cadena de frío en laboratorios de biología molecular, contribuyendo al cumplimiento normativo, la preservación de reactivos termosensibles y la mejora de la calidad diagnóstica.

7.2 Trabajo futuro

- Alertas por canales adicionales: Notificaciones por SMS, WhatsApp y correo electrónico ante excursiones de temperatura.
- Integración con LIMS: Conexión con el Laboratory Information Management System para correlacionar datos de temperatura con lotes de reactivos.
- Monitoreo de humedad relativa: Incorporación de sensores DHT22 para áreas con requisitos de humedad.
- Predicción de fallas mediante machine learning: Modelos predictivos basados en datos históricos para anticipar fallas en equipos de frío.
- Aplicación móvil: Consulta de estado y recepción de alertas desde dispositivos móviles.
- Expansión a otros laboratorios: Adaptación para despliegue en otros laboratorios o áreas del hospital.
- Certificación ante ANMAT: Evaluación de viabilidad de certificación como dispositivo de monitoreo.
- Redundancia de servidor: Esquema de failover automático con servidor secundario.

Alertas por canales adicionales: Notificaciones por SMS, WhatsApp y correo electrónico ante excursiones de temperatura.

Integración con LIMS: Conexión con el Laboratory Information Management System para correlacionar datos de temperatura con lotes de reactivos.

Monitoreo de humedad relativa: Incorporación de sensores DHT22 para áreas con requisitos de humedad.

Predicción de fallas mediante machine learning: Modelos predictivos basados en datos históricos para anticipar fallas en equipos de frío.

Aplicación móvil: Consulta de estado y recepción de alertas desde dispositivos móviles.

Expansión a otros laboratorios: Adaptación para despliegue en otros laboratorios o áreas del hospital.

Certificación ante ANMAT: Evaluación de viabilidad de certificación como dispositivo de monitoreo.

Redundancia de servidor: Esquema de failover automático con servidor secundario.

Referencias Bibliográficas

- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347-2376. <https://doi.org/10.1109/COMST.2015.2444095>
- ANMAT. (2004). Disposición 2819/2004. Requisitos de habilitación y funcionamiento para laboratorios de análisis clínicos. Administración Nacional de Medicamentos, Alimentos y Tecnología Médica, Argentina.
- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787-2805. <https://doi.org/10.1016/j.comnet.2010.05.010>
- Baker, S. B., Xiang, W., & Atkinson, I. (2017). Internet of Things for Smart Healthcare: Technologies, Challenges, and Opportunities. *IEEE Access*, 5, 26521-26544. <https://doi.org/10.1109/ACCESS.2017.2775180>
- Baust, J. G., Gao, D., & Baust, J. M. (2009). Cryopreservation: An emerging paradigm change. *Organogenesis*, 5(3), 90-96. <https://doi.org/10.4161/org.5.3.10021>
- CDC. (2019). Guidelines for Specimen Collection, Handling, and Transport. Centers for Disease Control and Prevention, U.S. Department of Health and Human Services. <https://www.cdc.gov/laboratory>
- Espressif Systems. (2020). ESP8266EX Datasheet. Version 6.6. https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf
- Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455. Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC6455>
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645-1660. <https://doi.org/10.1016/j.future.2013.01.010>
- ISO. (2012). ISO 15189:2012. Medical laboratories — Requirements for quality and competence. International Organization for Standardization, Geneva.
- Kodali, R. K., & Mahesh, K. S. (2016). A low cost implementation of MQTT using ESP8266. 2016 2nd International Conference on Contemporary Computing and Informatics (IC3I), 404-408. <https://doi.org/10.1109/IC3I.2016.7917998>

Maxim Integrated. (2019). DS18B20 Programmable Resolution 1-Wire Digital Thermometer. Datasheet. <https://datasheets.maximintegrated.com/en/ds/DS18B20.pdf>

Montanari, R., Bottani, E., Volpi, A., & Tebaldi, L. (2018). An IoT-based cold chain monitoring system for the pharmaceutical industry. *IFAC-PapersOnLine*, 51(11), 1612-1617. <https://doi.org/10.1016/j.ifacol.2018.08.270>

OASIS. (2019). MQTT Version 5.0. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

OMS. (2015). Directrices sobre la gestión de la cadena de frío. Organización Mundial de la Salud, Ginebra.

Ray, P. P. (2018). A survey on Internet of Things architectures. *Journal of King Saud University - Computer and Information Sciences*, 30(3), 291-319. <https://doi.org/10.1016/j.jksuci.2017.04.005>

Sambrook, J., & Russell, D. W. (2001). *Molecular Cloning: A Laboratory Manual* (3rd ed.). Cold Spring Harbor Laboratory Press, New York.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80-83. <https://doi.org/10.1109/MIC.2010.145>

Referencias adicionales sugeridas

[NOTA: Verificar disponibilidad y relevancia antes de incluir]

Ashton, K. (2009). That 'Internet of Things' Thing. *RFID Journal*, 22(7), 97-114.

OCDE. (1998). Principios de Buenas Prácticas de Laboratorio y verificación del cumplimiento. Serie de la OCDE sobre BPL, N° 1. París.

Sethi, P., & Sarangi, S. R. (2017). Internet of Things: Architectures, Protocols, and Applications. *Journal of Electrical and Computer Engineering*, 2017, Article 9324035. <https://doi.org/10.1155/2017/9324035>

WHO. (2015). WHO Technical Report Series, No. 992. Annex 9: Model guidance for the storage and transport of time- and temperature-sensitive pharmaceutical products. Geneva.

Xu, L. D., He, W., & Li, S. (2014). Internet of Things in Industries: A Survey. *IEEE Transactions on Industrial Informatics*, 10(4), 2233-2243. <https://doi.org/10.1109/TII.2014.2300753>

Beck, K., et al. (2001). Manifesto for Agile Software Development. Retrieved from <https://agilemanifesto.org/>

Larman, C., & Basili, V. R. (2003). Iterative and incremental developments: A brief history. *Computer*, 36(6), 47-56.

Mineraud, J., Mazhelis, O., Su, X., & Tarkoma, S. (2016). A gap analysis of Internet-of-Things platforms. *Computer Communications*, 89, 5-16.

Patel, P., & Cassou, D. (2015). Enabling high-level application development for the Internet of Things. *Journal of Systems and Software*, 103, 62-84.

Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26(August), 1-9.

Anexo A: Código Fuente del Firmware ESP8266 y Esquema Electrónico

Se presentan los fragmentos más representativos del firmware modular SmartTemp y el esquema de conexión electrónica.

A.1 Hardware Implementado



Figura A.1. Sensor DS18B20 conectado a ESP8266 NodeMCU



Figura A.2. Datalogger LogTag UTRID-16 para validación de precisión

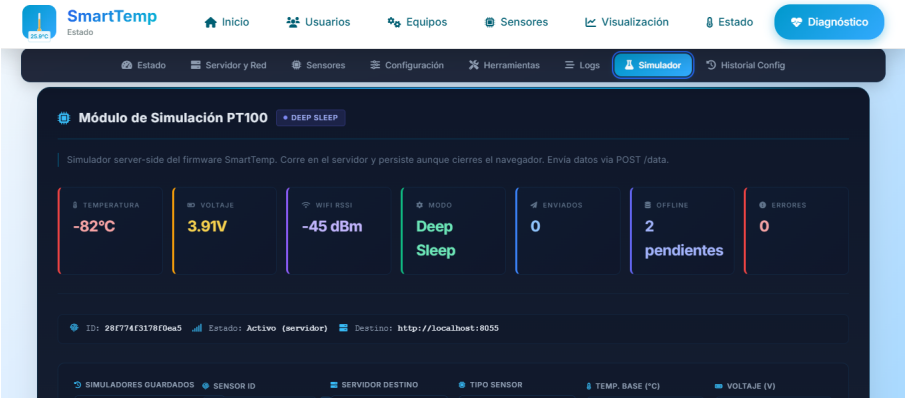


Figura A.3. Sensor virtual implementado para pruebas y desarrollo

A.2 Esquema de Conexión Electrónica

[Diagrama: ver versión HTML]

Pin ESP8266	Función	Componente
GPIO14 (D5)	OneWire Data	DS18B20 (DQ)
A0 (ADC)	Lectura voltaje	Divisor de voltaje
3.3V	Alimentación	DS18B20 (VDD), Pull-up 4.7k:•
GND	Tierra	DS18B20 (GND), Divisor

A.3 Código Fuente Principal

funcional_remoto_modular_.ino

```

#include <WiFiClient.h>
#include <ESP8266HTTPClient.h>
#include <ESP8266WebServer.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ArduinoJson.h>
#include <EEPROM.h>
#include <PubSubClient.h>
#include <user_interface.h>
#include <LittleFS.h>
#include <WiFiUdp.h>
#include <time.h>

#include "config.h"
#include "types.h"
#include "sensor.h"
#include "storage.h"
#include "wifi_manager.h"
#include "mqtt_manager.h"
#include "web_server.h"
#include "mode_manager.h"
#include "sensor_fix.h"

// ===== VARIABLES GLOBALES =====
const char* my_ssid = WIFI_SSID;
const char* my_password = WIFI_PASSWORD;
const char* serverIP = SERVER_IP;
const uint16_t serverPort = SERVER_PORT;
const char* API_ENDPOINT_PATH = API_ENDPOINT;

// ... (295 líneas omitidas)

```

mqtt_manager.cpp

```

#include "mqtt_manager.h"
#include "config.h"
#include "sensor.h"
#include "storage.h"

// ===== MQTT: Helper para suscribir topics =====
void mqttSubscribeAll() {
    mqttClient.subscribe("sensores/ping/request");
    mqttClient.subscribe(MQTT_ACK_TOPIC);
    mqttClient.subscribe(MQTT_TIME_RESPONSE);
    String configTopic = "sensores/config/" + sensorID;
    mqttClient.subscribe(configTopic.c_str());
    String resetTopic = "sensores/reset/" + sensorID;
    mqttClient.subscribe(resetTopic.c_str());
}

// ===== MQTT: Intentar conectar a un servidor con N reintentos =====
bool tryMQTTServer(const char* server, int port, const char* label, int retries) {
    String clientId = "ESP8266_" + sensorID;

    for (int i = 1; i <= retries; i++) {
        mqttClient.disconnect();
        espClient.stop();
        espClient = WiFiClient();
        mqttClient.setClient(espClient);
        mqttClient.setServer(server, port);

        Serial.printf("MQTT %s %s:%d (intento %d/%d)\n", label, server, port, i, retries);

        if (mqttClient.connect(clientId.c_str())) {
            // ... (394 líneas omitidas)

```

sensor_core.cpp

```
#include "sensor.h"
#include "config.h"
#include "storage.h"
#include <EEPROM.h>

const float VOLTAGE_EMA_ALPHA = 0.3;
bool sensorChanged = false;
String previousSensorID = "";

// initSensor: Inicializa sensor Dallas DS18B20.
// En deep sleep: solo carga IDs de EEPROM sin encender hardware (ahorro bateria).
// En continuo: escanea bus OneWire completo y registra sensores fisicos.
// readSensorOptimized() es el unico que enciende/apaga el sensor en deep sleep.
void initSensor() {
    bool skipHardwareScan = (currentMode == MODE_DEEP_SLEEP);

    if (!skipHardwareScan) {
        // Modo continuo: escaneo completo del bus OneWire
        digitalWrite(POWER_PIN, HIGH);
        delay(SENSOR_POWER_UP_DELAY);

        sensors.begin();
        sensorCount = sensors.getDeviceCount();
        Serial.printf("Sensores encontrados: %d\n", sensorCount);

        if (sensorCount > MAX_CHANNELS) sensorCount = MAX_CHANNELS;

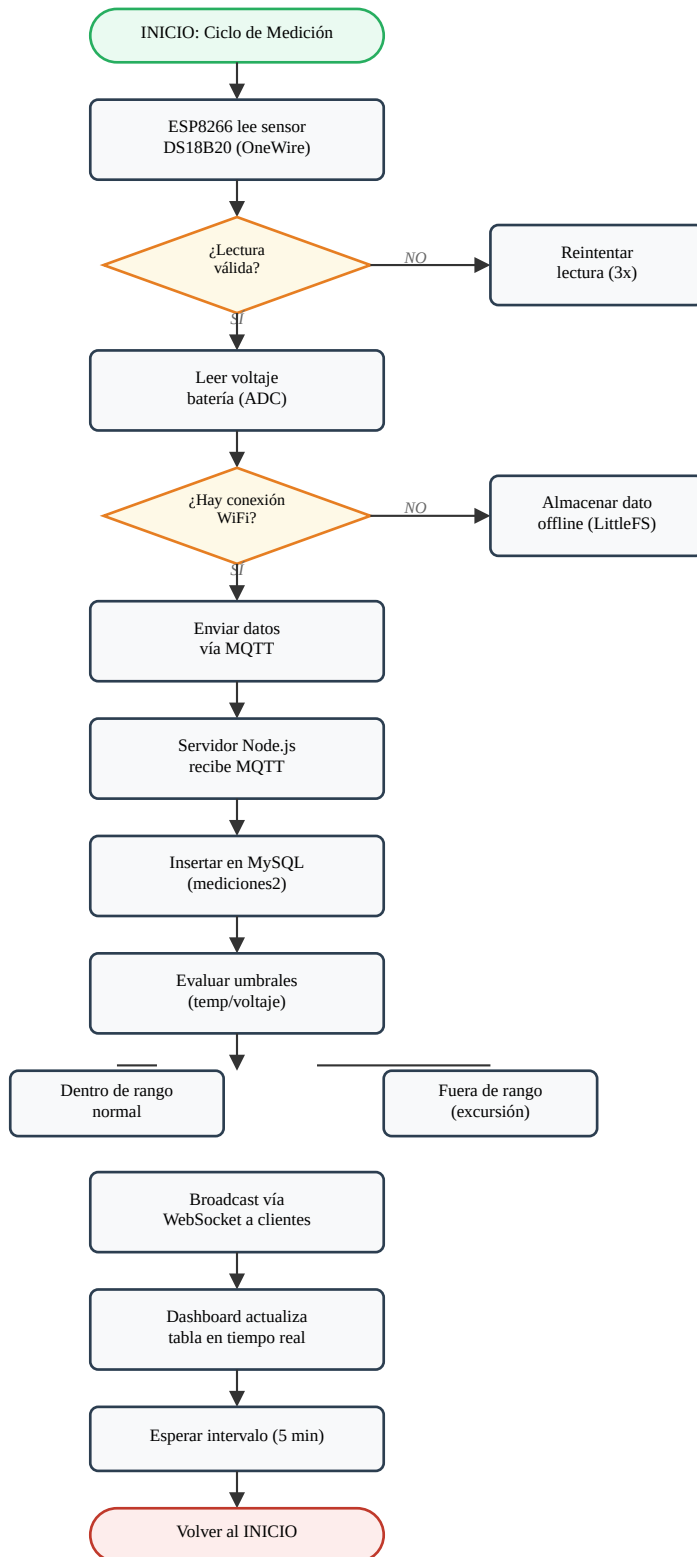
        for (int i = 0; i < sensorCount; i++) {
            if (sensors.getAddress(sensorAddresses[i], i)) {
                sensors.setResolution(sensorAddresses[i], SENSOR_RESOLUTION);
            }
        }
    }
    // ... (200 líneas omitidas)
```

Anexo B: Diagramas de Flujo del Proceso y Arquitectura del Sistema

Diagramas de flujo de los procesos principales del sistema SmartTemp, esquema de base de datos y diagramas UML.

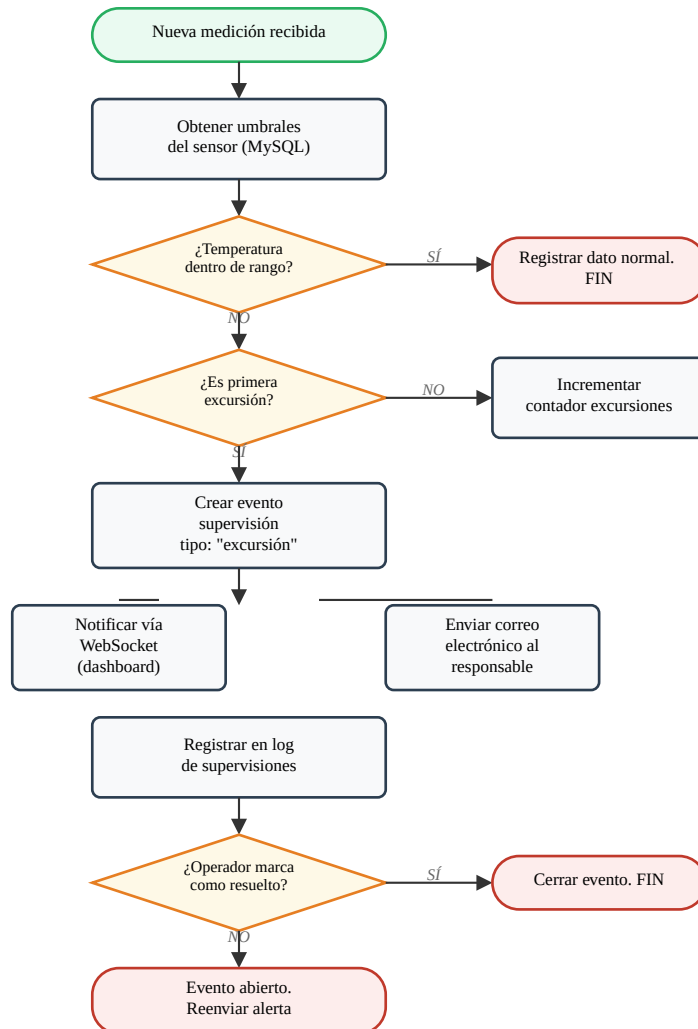
B.1 Flujo Principal: Adquisición y Visualización de Datos

Diagrama de flujo del proceso completo desde la lectura del sensor hasta la visualización en el dashboard.



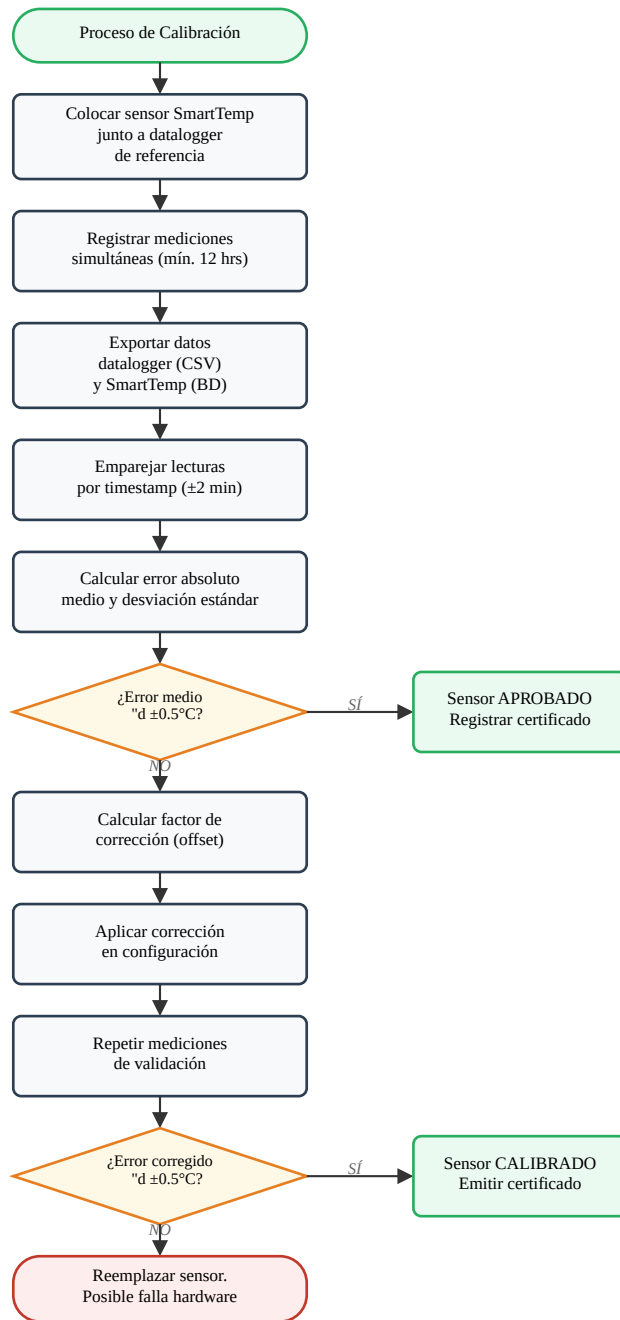
B.2 Flujo de Alertas por Excursión de Temperatura

Proceso de detección y notificación cuando un sensor registra valores fuera de los umbrales configurados.



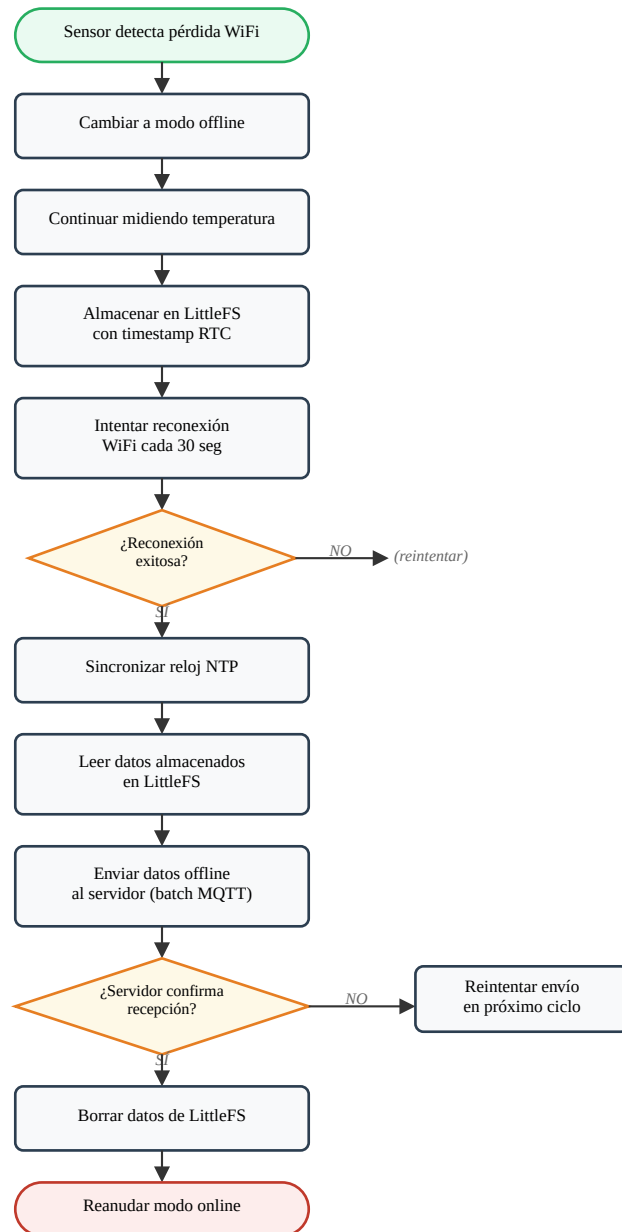
B.3 Flujo de Calibración de Sensores

Proceso de calibración comparando lecturas del sensor SmartTemp con un datalogger de referencia certificado.



B.4 Flujo de Reconexión y Datos Offline

Proceso de manejo de desconexiones WiFi y envío de datos almacenados localmente.



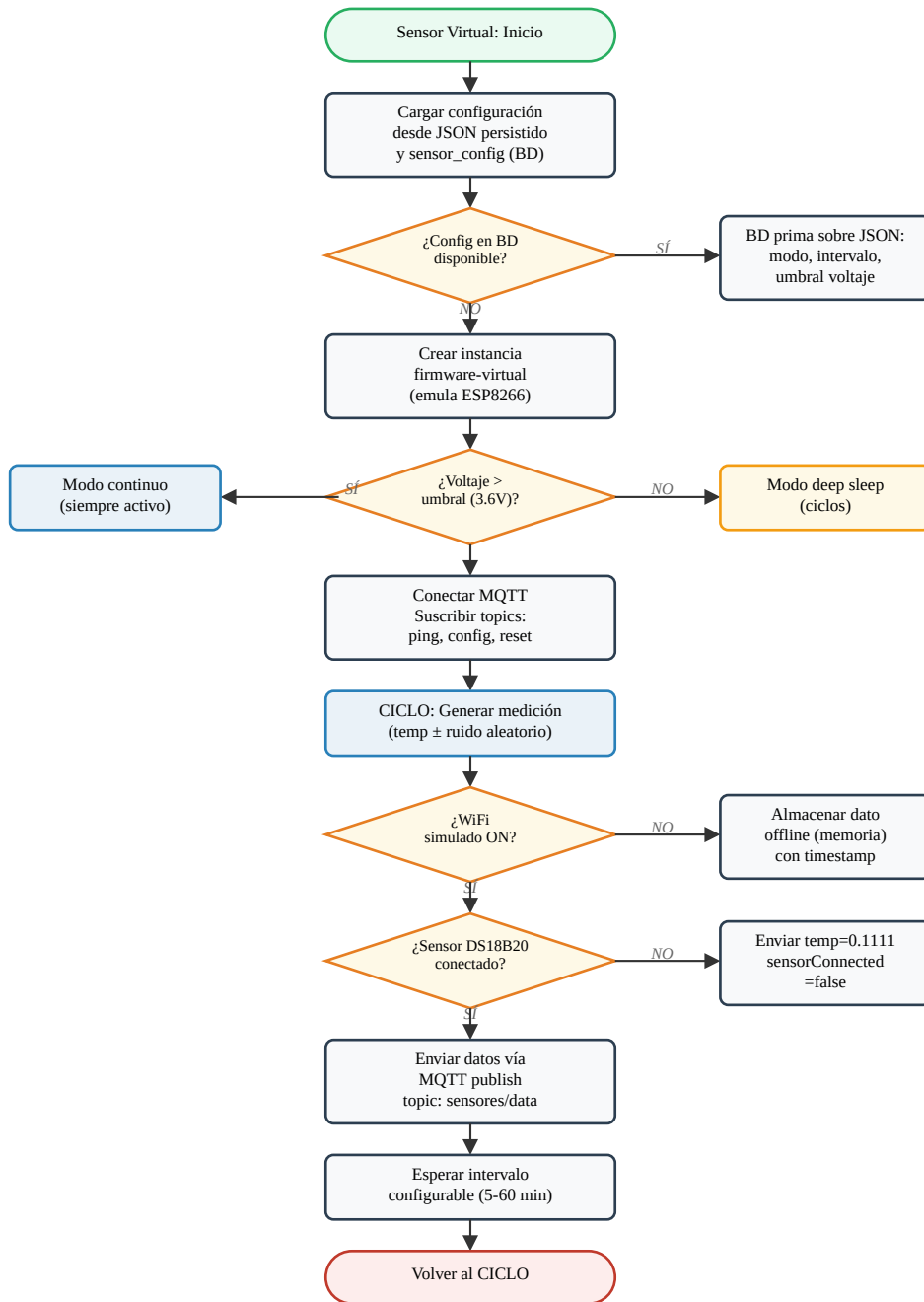
B.5 Esquema de Base de Datos MySQL

El sistema SmartTemp utiliza una base de datos MySQL con las siguientes tablas principales:

Tabla	Función	Campos principales
mediciones2	Datos de sensores	idsensor, temperatura, voltaje, timestamp
sensores	Configuración sensores	idsensor, nombre, ubicacion, activo
equipos	Equipos monitoreados	idequipo, nombre, tipo, sector
umbrales	Límites de temperatura	idsensor, temp_max, temp_min
supervisiones	Eventos y alertas	idsensor, tipo, timestamp, resuelto
usuarios	Autenticación	idusuario, username, password_hash

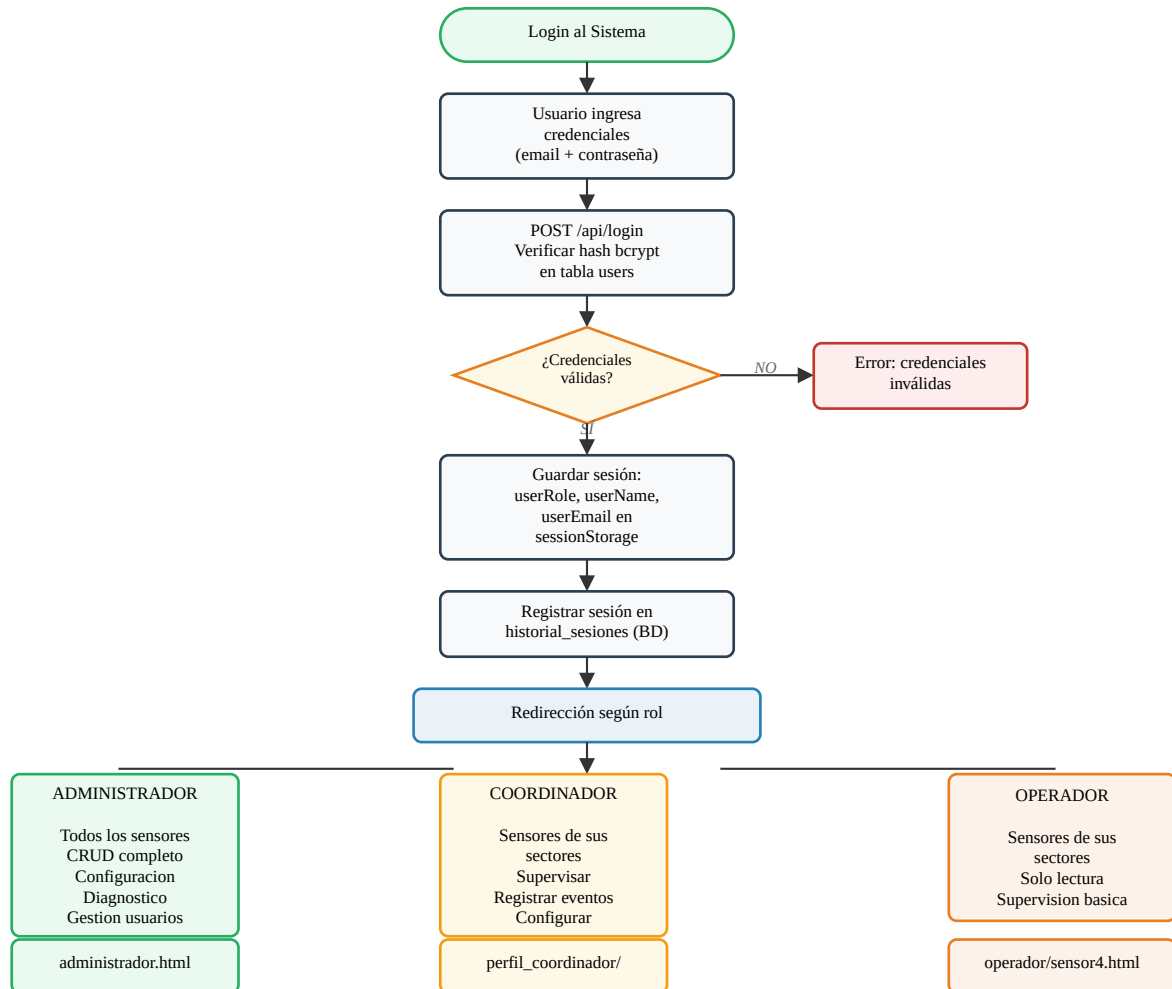
B.6 Flujo del Sensor Virtual / Simulador

Proceso del sensor virtual que emula el comportamiento del firmware ESP8266 en software, permitiendo pruebas sin hardware físico.



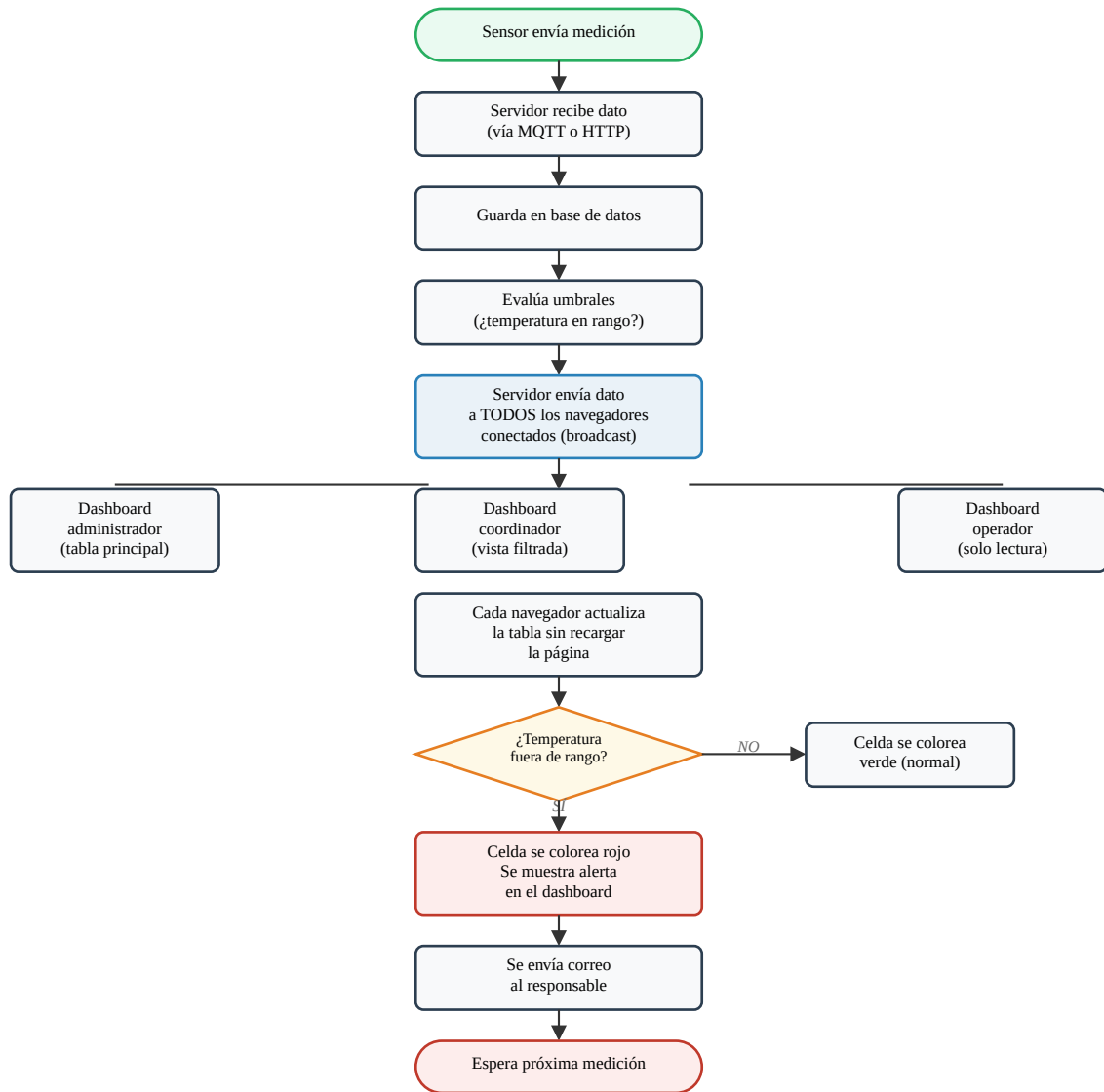
B.7 Login y Redirección por Perfil

Proceso de autenticación y redirección según el rol del usuario, con detalle de permisos por perfil.



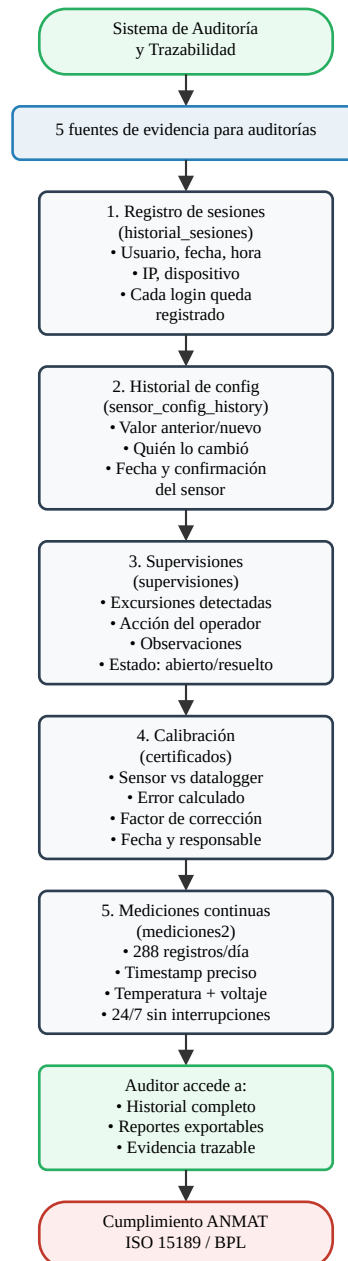
B.8 Flujo de Comunicación WebSocket

Flujo simplificado de cómo el servidor distribuye los datos de temperatura a todos los navegadores conectados en tiempo real.



B.9 Sistema de Auditoría y Trazabilidad

Fuentes de evidencia que el sistema genera automáticamente para cumplimiento normativo (ANMAT, ISO 15189, BPL).



B.10 Diagrama de Clases UML (Simplificado)

[Diagrama: ver versión HTML]

B.11 Diagrama de Secuencia - Envío de Datos

[Diagrama: ver versión HTML]

Anexo C: Esquema de Base de Datos y Diagramas UML

Esquema consolidado de la base de datos MySQL y diagramas UML del sistema SmartTemp.

C.1 Tablas Principales de la Base de Datos

Base de Datos MySQL

Descripción General

Funcionamiento

Campo	Tipo	Descripción
id	INT (PK, AUTO_INCREMENT)	Identificador único
nombre	VARCHAR	Nombre del usuario
correo_electronico	VARCHAR	Email, se usa para login
password	Vexto plano actualmente)	
Rol	VARCHAR	`administrador`, `coordinador`, `operador`, `visualizador`
ultimo_ingreso	DATETIME	Fecha/hora del último login

Flujo de Datos

C.2 Diagramas UML del Sistema

Diagrama de Clases Principal

[Diagrama: ver versión HTML]

Diagrama de Secuencia - Envío de Datos

[Diagrama: ver versión HTML]

Anexo D: Capturas de Pantalla del Sistema

Capturas de pantalla principales del sistema SmartTemp.

D.1 Interfaz Principal

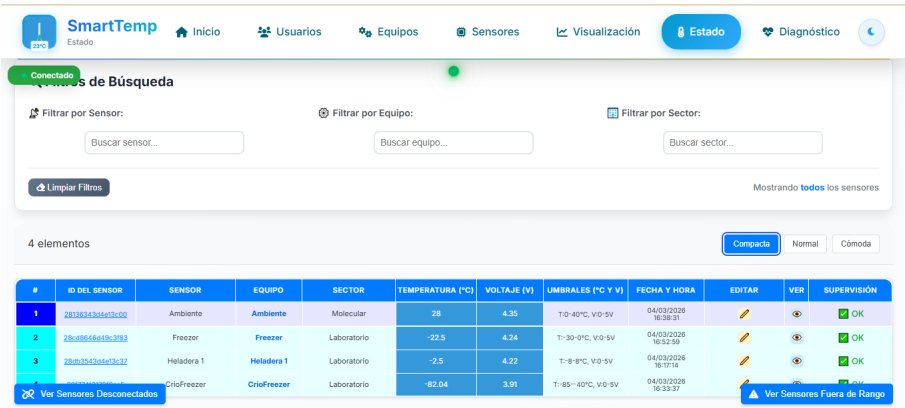


Figura D.1. Interfaz principal con mediciones en tiempo real

D.2 Panel de Administración



Figura D.2. Panel de administración del sistema

D.3 Configuración de Sensores

The screenshot shows the 'SmartTemp' web interface with the 'Configuración' (Configuration) tab selected. The main section is titled 'Envío Paralelo de Configuraciones' (Parallel Configuration Sending) and includes a description: 'Envía las 3 configuraciones en un solo mensaje (bundle)'. Below this, there are several configuration options:

- Modo:** A dropdown menu set to 'Por Sensor'.
- Sensor:** A dropdown menu labeled 'Seleccionar sensor...'.
- Filtrar:** A search input field labeled 'Buscar sensor...'.
- Intervalo Medición:** A dropdown menu set to 'No cambiar'.
- Intervalo UPS:** A dropdown menu set to 'No cambiar'.
- Intervalo Continuo:** A dropdown menu set to 'No cambiar'.
- Umbral: Modo de medición:** A dropdown menu set to 'No cambiar'.
- Modo:** A dropdown menu set to 'No cambiar'.

At the bottom left, there is a purple button labeled 'Enviar Todo'. At the bottom right, there is a grey button labeled 'Selecciona un sensor y las configuraciones a enviar'.

Figura D.3. Configuración general y umbrales de sensores

D.4 Sensor Virtual

The screenshot shows the 'SmartTemp' web interface with the 'Simulador' (Simulator) tab selected. The main section is titled 'Módulo de Simulación PT100' and includes a sub-header 'DEEP SLEEP'. Below this, there is a description: 'Simulador server-side del firmware SmartTemp. Corre en el servidor y persiste aunque cierres el navegador. Envía datos via POST /data.'.

The simulator displays several key metrics in a grid:

- TEMPERATURA:** -82°C
- VOLTAJE:** 3.91V
- WIFI RSSI:** -45 dBm
- MODO:** Deep Sleep
- ENVIADOS:** 0
- OFFLINE:** 2 pendientes
- ERRORES:** 0

Below the grid, there is a status bar showing: ID: 28f774f3178f0ea5, Estado: Activo (servidor), Destino: http://localhost:8055.

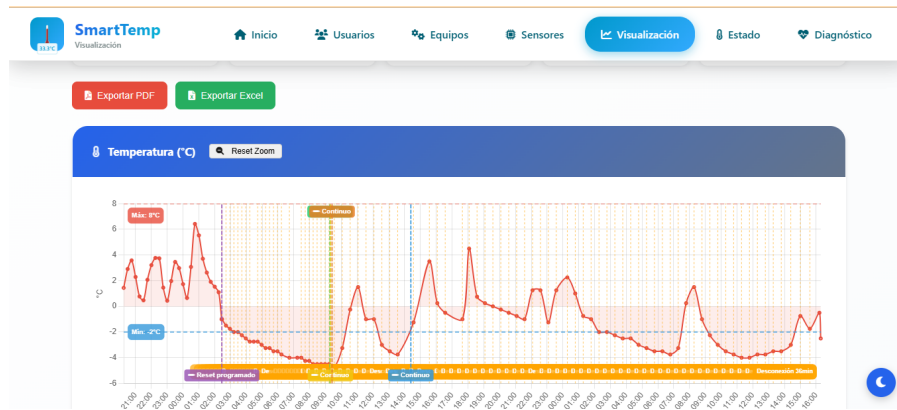
At the bottom, there is a navigation bar with several tabs: SIMULADORES GUARDADOS, SENSOR ID, SERVIDOR DESTINO, TIPO SENSOR, TEMP. BASE (°C), and VOLTAJE (V).

Figura D.4. Panel del sensor virtual y simulador

Anexo E: Capturas de Pantalla del Sistema

Capturas adicionales del sistema SmartTemp en funcionamiento.

Figura E.1: Gráficos de Temperatura



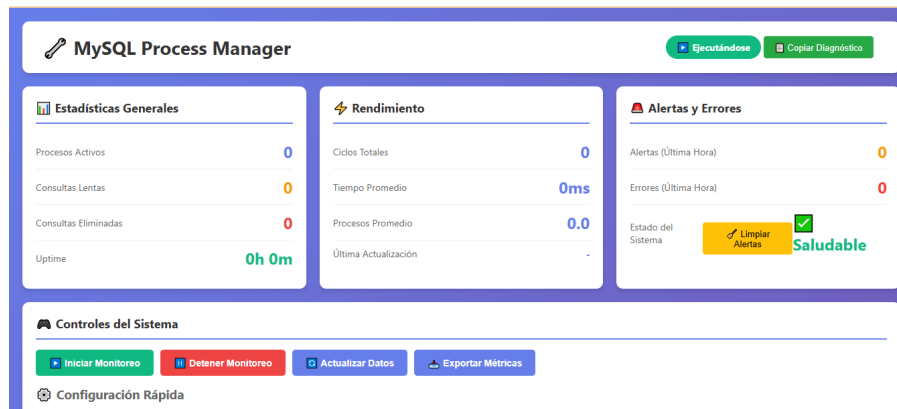
Gráficos de series temporales mostrando la evolución de temperatura por equipo.

Figura E.2: Módulo de Calibración

The screenshot displays the SmartTemp interface with a navigation bar at the top containing links for Inicio, Usuarios, Equipos, Sensores, Visualización, Estado (active), and Diagnóstico. Below the navigation bar is a green bar indicating 'Conectado' and 'Conexión con Patrón'. The main area is divided into two sections: 'PATRÓN DE REFERENCIA' and 'RESPONSABLE'. The 'PATRÓN DE REFERENCIA' section contains fields for Nombre (Temp), Marca (SmartTemper), Modelo (SmartTemper v1), N° Serie (198502), Calibración (01/04/2026), and Caducidad (01/04/2027). The 'RESPONSABLE' section contains fields for Nombre (Ing. Nombre), Cargo (Ing. Biomédico), Fecha (dd/mm/aaaa), and Firma (PNG). There is a button 'Seleccionar archivo' and a text 'Ningún archivo seleccionado'.

Interfaz de calibración de sensores con factores de corrección.

Figura E.3: Sistema de Diagnóstico



Panel de diagnóstico del sistema mostrando estado de conexiones y métricas.

Figura E.4: Ajuste de Umbrales



Módulo de configuración de umbrales de temperatura para alertas automáticas.

Anexo F: Manual de Operación del Sistema

Manual de operación del sistema SmartTemp para el operador del laboratorio. Las capturas de pantalla de las interfaces se encuentran en los Anexos D y E.

Sistema de Diagnóstico

Sistema de Diagnóstico

> Panel de control técnico para verificar el estado del servidor, red, sensores, interfaces de red y configurar intervalos de envío de datos.

Descripción General

La página de diagnóstico (`diagnostico/index.html`) es el panel de control técnico del sistema SmartTemp. Permite verificar en tiempo real el estado de todos los componentes: servidor backend, conexión WebSocket, sensores individuales (ping HTTP y MQTT), interfaces de red, y configurar los intervalos de envío de datos.

En sistemas IoT de monitoreo continuo, la capacidad de diagnóstico remoto es fundamental para mantener la disponibilidad del servicio. La página de diagnóstico implementa el concepto de "observabilidad" (observability) de sistemas distribuidos, proporcionando tres pilares: métricas (estado del servidor, tiempos de respuesta), logs (registro de eventos y consola del servidor) y trazas (flujo de comunicación WebSocket y MQTT).

Funcionamiento

****1. Estado del Servidor**:** Indicador circular (verde/rojo/gris), verificación HTTP con `/ping` !' "pong".

****2. Estado de Red (WebSocket)**:** Estado de conexión, intentos de reconexión, mensajes enviados/recibidos, latencia ping/pong. Botones: Reconectar, Enviar Ping.

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Gestión de Equipos y Sectores

Gestión de Equipos y Sectores

> Jerarquía organizacional Sector !' Equipo !' Sensor: CRUD completo, asociaciones, auto-detección de sensores y eliminación en cascada.

Descripción General

SmartTemp organiza los sensores dentro de una estructura jerárquica de tres niveles que refleja la organización física del laboratorio. Esta jerarquía permite agrupar, filtrar y administrar los dispositivos de forma lógica, y es la base del sistema de control de acceso por roles.

Sector (ej: "Laboratorio Hematología") % % % Equipo (ej: "Heladera Reactivos A") % % % Sensor (ej: "28FF6A1234...") En sistemas de monitoreo de cadena de frío para laboratorios, la organización jerárquica es esencial para cumplir con los requisitos de trazabilidad de la norma ISO 15189. Cada punto de monitoreo (sensor) debe estar asociado a un equipo de almacenamiento específico (heladera, freezer) y a un área del laboratorio (sector), permitiendo generar reportes de cumplimiento por zona.

```
Sector (ej: "Laboratorio Hematología")
└─ Equipo (ej: "Heladera Reactivos A")
    └─ Sensor (ej: "28FF6A1234...")
```

Funcionamiento

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Supervisiones

Sistema de Supervisiones y Encuestas

> Registro de supervisiones cuando un sensor tiene problemas, y sistema de encuestas para evaluar el impacto del sistema en el personal del laboratorio.

Descripción General

SmartTemp incluye dos sistemas de registro de información humana: las supervisiones (cuando un usuario revisa un sensor con problemas) y las encuestas (formularios para el personal del laboratorio). Ambos sistemas son fundamentales para la trazabilidad y la evaluación del impacto del sistema.

Las supervisiones implementan un flujo de gestión de incidencias inspirado en los principios de ITIL (Information Technology Infrastructure Library) para gestión de servicios. Cuando un sensor presenta una anomalía (desconexión, temperatura fuera de rango), el sistema genera una alerta visual y permite al personal registrar que revisó el problema, qué observó y si lo resolvió. Esto crea un registro auditable de las acciones tomadas ante cada incidencia.

Las encuestas fueron diseñadas específicamente para la tesis de maestría, permitiendo evaluar cuantitativamente el impacto del sistema de monitoreo IoT comparando la situación antes y después de la implementación.

Funcionamiento

****¿Cuándo aparece el botón de supervisar?****

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Anexo G: Manual de Instalación y Configuración

Guía de instalación del sistema SmartTemp: servidor Node.js, broker MQTT, MySQL y sensores ESP8266.

Despliegue e Instalación del Servidor

Despliegue e Instalación del Sistema

> Instalación, configuración de entornos, gestión de procesos con PM2, sistema de logs y despliegue en producción.

Descripción General

SmartTemp opera en dos entornos: desarrollo local (red del laboratorio) y producción (servidor remoto accesible desde internet). El despliegue en producción utiliza PM2 como gestor de procesos para garantizar disponibilidad continua, con auto-restart ante fallos y gestión de logs.

La estrategia de despliegue sigue las mejores prácticas para aplicaciones Node.js en producción: separación de configuración por entorno (variables de entorno con dotenv), gestión de procesos con PM2 (equivalente a systemd para Node.js), y monitoreo de recursos. PM2 es el gestor de procesos más utilizado para Node.js en producción, con más de 40 millones de descargas mensuales en npm.

Funcionamiento

Entorno	IP	Puerto	Uso
---------	----	--------	-----

-----	-----	-----	-----
-------	-------	-------	-------

Desarrollo	192.168.1.24	8055	Pruebas en red local del laboratorio
------------	--------------	------	--------------------------------------

Producción	149.50.137.2	8055	Servidor accesible desde internet
------------	--------------	------	-----------------------------------

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Configuración del Broker MQTT

Sistema MQTT

> Comunicación bidireccional entre servidor y sensores ESP8266 mediante el protocolo MQTT: ping remoto, configuración de intervalos y broker Mosquitto.

Descripción General

MQTT (Message Queuing Telemetry Transport) es el protocolo que permite al servidor comunicarse con los sensores ESP8266 de forma bidireccional, incluso cuando los sensores no son accesibles directamente por red HTTP. Fue creado por IBM en 1999 y estandarizado por OASIS en 2014, convirtiéndose en el estándar de facto para comunicación IoT.

A diferencia de HTTP (modelo petición-respuesta donde el cliente siempre inicia), MQTT usa un modelo de publicación/suscripción con un intermediario (broker). Esto permite que el servidor envíe comandos a los sensores en cualquier momento sin que estos tengan que estar "escuchando" peticiones HTTP. El overhead del protocolo es mínimo (cabecera de 2 bytes vs ~700 bytes de HTTP), lo que lo hace ideal para dispositivos con recursos limitados como el ESP8266.

SmartTemp usa MQTT para tres funciones que no son posibles con HTTP:

1. ****Ping remoto****: Verificar si un sensor está activo
2. ****Configuración remota****: Cambiar intervalos de envío sin acceso físico
3. ****Comunicación bidireccional****: El servidor puede enviar comandos a los sensores

Funcionamiento

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Autenticación y Seguridad

Sistema de Autenticación y Login

> Proceso de inicio de sesión, gestión de sesiones en el navegador, roles de usuario y registro de historial de accesos.

Descripción General

SmartTemp implementa un sistema de autenticación basado en sesiones del lado del cliente (localStorage/sessionStorage) con verificación de credenciales contra la base de datos MySQL. El sistema soporta múltiples roles con diferentes niveles de acceso y registra un historial completo de inicios de sesión para auditoría.

La autenticación es un componente crítico en sistemas IoT de monitoreo, ya que controla quién puede visualizar datos sensibles de la cadena de frío y quién puede modificar configuraciones de los sensores. En el contexto de la norma ISO 15189 para laboratorios clínicos, la trazabilidad de accesos es un requisito para acreditación.

La página de login es el punto de entrada principal del sistema, accesible desde la raíz del servidor (`/index.html`).

Funcionamiento

1. Al abrir la página de login, el sistema carga la lista de usuarios desde `GET /api/users`
2. El usuario ingresa correo electrónico y contraseña

Contenido resumido. Ver documentación completa en el repositorio del proyecto.

Anexo H: Resultados de Pruebas de Precisión y Gráficos Temporales

Comparación de precisión entre datalogger de referencia y sensores SmartTemp, con gráficos de series temporales.

H.1 Estadísticas de Precisión

Datos de cruce datalogger no disponibles.

H.2 Gráficos de Series Temporales

Datos de series temporales no disponibles.

Anexo I: Guía de Configuración Avanzada

Guía completa para la instalación y configuración del sistema SmartTemp.

I.1 Requisitos del Sistema

Componente	Especificación
Servidor	Node.js 16+, MySQL 8.0+, PM2
Hardware	ESP8266 NodeMCU, DS18B20, resistencias
Red	WiFi 2.4GHz, acceso a internet (opcional)
Navegador	Chrome 90+, Firefox 88+, Safari 14+

I.2 Instalación del Servidor

```
# 1. Clonar repositorio
git clone https://github.com/usuario/smarttemp.git
cd smarttemp

# 2. Instalar dependencias
npm install

# 3. Configurar base de datos
mysql -u root -p < db-scripts/setup-db.js

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con credenciales MySQL

# 5. Iniciar servidor
npm start
# o con PM2: pm2 start ecosystem.config.js
```

I.3 Configuración de Sensores ESP8266

```
// config.h - Configuración WiFi y servidor
#define WIFI_SSID "NombreRed"
#define WIFI_PASSWORD "ClaveRed"
#define SERVER_HOST "192.168.1.100"
#define SERVER_PORT 3000
#define SENSOR_ID "28db3543d4e13c37"

// Compilar y subir firmware
// Arduino IDE: Herramientas > Placa > ESP8266 > NodeMCU 1.0
```

I.4 Verificación de Funcionamiento

- Acceder a <http://servidor:3000>
- Verificar conexión de sensores en dashboard
- Configurar umbrales de temperatura
- Probar alertas y notificaciones
- Validar registro histórico de datos

Anexo K: Presupuesto del Sistema y Normativa Aplicable

Análisis de costos del sistema SmartTemp y marco normativo aplicable.

K.1 Presupuesto del Sistema SmartTemp

Bill of Materials (BOM) por nodo sensor

Componente	Descripción	Costo (USD)
ESP8266 (Wemos D1 Mini)	Microcontrolador WiFi 802.11 b/g/n	~2.0
DS18B20 (encapsulado)	Sensor temperatura digital, cable 1m	~1.0
Resistencias (4.7k, 10k, 330k)	Pull-up OneWire + divisor voltaje	~0.0
Fuente 5V USB + cable	Adaptador pared + cable micro-USB	~2.5
Carcasa + PCB	Caja IP54 + placa montaje	~1.5
Total por nodo		~7.0

Costos de infraestructura (mensual)

Concepto	Descripción	Costo (USD/mes)
VPS servidor	1 vCPU, 1GB RAM, 25GB SSD	~5.0
Dominio	Registro .com (anualizado)	~1.0
MQTT + MySQL + SSL	Incluido en VPS	0.0
Total mensual		~6.0

Comparación con soluciones comerciales

Solución	Costo inicial	Costo mensual	Observaciones
SmartTemp (propuesta)	\$70 (10 sensores)	\$6	Código abierto, personalizable
Testo Saveris 2	\$1,200-2,000	\$50-100	Solución comercial completa
Onset HOBO	\$800-1,500	\$30-60	Dataloggers + software

K.2 Marco Normativo Aplicable

Normativas Nacionales (Argentina)

Norma	Descripción	Aplicabilidad SmartTemp
ANMAT Disp. 2819/2004	Buenas Prácticas de Laboratorio	Trazabilidad condiciones ambientales
ANMAT Disp. 3602/2004	Habilitación laboratorios	Requisitos infraestructura
Ley 25.3	Protección Datos Personales	Manejo información laboratorio

Normativas Internacionales

Norma	Descripción	Requisitos SmartTemp
ISO 15189:2012	Laboratorios clínicos	Control condiciones ambientales
ISO/IEC 17025:2017	Competencia laboratorios	Trazabilidad mediciones
WHO TRS 961	Cadena frío vacunas	Monitoreo continuo temperatura
FDA 21 CFR Part 11	Registros electrónicos	Integridad datos digitales

Requisitos Técnicos Específicos

- Precisión: $\pm 0.5^{\circ}\text{C}$ (ISO 15189, ANMAT 2819/2004)
- Frecuencia: Registro continuo cada 5-15 minutos
- Trazabilidad: Timestamps precisos, registros inalterables
- Alertas: Notificación inmediata de excursiones
- Respaldo: Almacenamiento redundante de datos
- Calibración: Verificación anual con patrones certificados

Cumplimiento SmartTemp

El sistema SmartTemp cumple con los requisitos normativos mediante:

- Precisión $\pm 0.5^{\circ}\text{C}$ validada con datalogger calibrado
- Registro continuo cada 5 minutos con timestamps UTC
- Base de datos MySQL con integridad referencial
- Alertas automáticas por email y WebSocket
- Respaldo automático de datos y logs de auditoría
- Interfaz de calibración con factores de corrección

Anexo L: Normativa Aplicable

ANMAT — Disposición 2819/2004

La Administración Nacional de Medicamentos, Alimentos y Tecnología Médica (ANMAT) establece en la Disposición 2819/2004 los requisitos para las Buenas Prácticas de Laboratorio (BPL) en laboratorios de análisis clínicos y bioquímicos.

Requisitos relevantes para monitoreo de temperatura

- Los equipos de conservación de muestras y reactivos deben contar con sistemas de monitoreo de temperatura que garanticen el mantenimiento de las condiciones especificadas.
- Se deben mantener registros de temperatura de los equipos de almacenamiento, con frecuencia adecuada para detectar desviaciones.
- Los registros de temperatura deben ser trazables y estar disponibles para auditorías.
- Se deben establecer procedimientos de acción correctiva ante excursiones de temperatura.
- Los instrumentos de medición deben estar calibrados contra patrones de referencia trazables.

ISO 15189:2012 — Laboratorios clínicos, §5.3 Equipamiento de laboratorio

La norma ISO 15189:2012 establece los requisitos de calidad y competencia para laboratorios clínicos. La sección 5.3 aborda específicamente el equipamiento de laboratorio.

§5.3.1 Generalidades

- El laboratorio debe disponer de equipos necesarios para la prestación de servicios, incluyendo equipos de monitoreo ambiental y de almacenamiento.
- Los equipos deben ser capaces de alcanzar el rendimiento requerido y cumplir con las especificaciones pertinentes.

§5.3.2 Aceptación del equipamiento

- Antes de su uso, los equipos deben ser verificados para confirmar que cumplen con los requisitos especificados, incluyendo precisión y exactitud de medición.

§5.3.3 Instrucciones de uso

- Las instrucciones de uso del equipamiento deben estar disponibles para el personal.

§5.3.4 Calibración y trazabilidad metrológica

- Los equipos de medición deben ser calibrados a intervalos regulares utilizando materiales de referencia certificados o patrones trazables al Sistema Internacional de Unidades (SI).
- Se deben mantener registros de calibración que incluyan: fecha, resultado, criterio de aceptación, y próxima fecha de calibración.

Principios de Buenas Prácticas de Laboratorio (BPL)

Los principios de BPL, establecidos por la OCDE y adoptados por ANMAT, definen un marco de calidad para la planificación, realización, monitoreo, registro, archivo y reporte de estudios de laboratorio.

Principios relevantes para el monitoreo de cadena de frío

- Trazabilidad: Todos los datos generados deben ser trazables desde su origen hasta el informe final, incluyendo registros de temperatura.
- Integridad de datos: Los registros electrónicos deben ser protegidos contra modificaciones no autorizadas, con pistas de auditoría.
- Calibración de instrumentos: Los instrumentos de medición deben ser calibrados regularmente contra estándares de referencia trazables.
- Documentación: Se deben mantener procedimientos operativos estándar (POE) para el monitoreo de temperatura, incluyendo frecuencia de registro, umbrales de alerta y acciones correctivas.
- Conservación de registros: Los registros de temperatura deben conservarse por el período establecido por la normativa aplicable (mínimo 5 años según ANMAT).
- Validación de sistemas computarizados: Los sistemas electrónicos de monitoreo deben ser validados para demostrar que cumplen con su propósito previsto, incluyendo precisión, disponibilidad y capacidad de almacenamiento.

